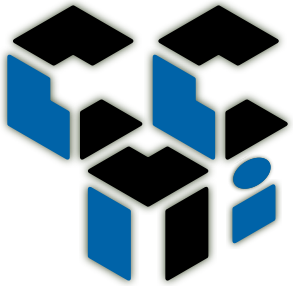
# OntoUML: Vztahy Celek-Část, typy agregací; Aspekty





FAKULTA
INFORMAČNÍCH
TECHNOLOGIÍ
ČVUT V PRAZE

**BI-KOM: Konceptuální modelování #4**

**doc. Ing. Robert Pergl, Ph.D. (robert.pergl@fit.cvut.cz)**

Pozn.: Pro přípravu přednášky byly mj. využity dostupné materiály prof. Giancarla Guizzardiho.

# Obsah přednášky

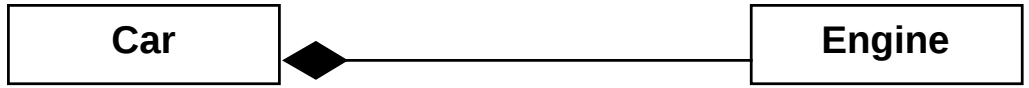
1. Teorie celku a částí
2. Povinnosti ve vztahu celek-část
3. `{essential}`, `{inseparable}`, `{immutablePart}`, `{immutableWhole}`
4. Typy agregací: «Quantity», «Collective», Funkční Celek
5. Aspekty: «Quality», «Mode»

# Teorie celku a části

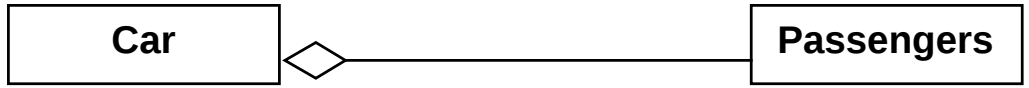
- Relace Celek-Část jsou klíčové z hlediska našeho vnímání (cognitive science), tj. pro realizaci řady důležitých kognitivních úkolů.
- Z tohoto důvodu jsou vztahy Celek-Část klíčové též ve většině metodik objektově-orientovaného modelování.
- Studiem vztahů Celek-Část se tradičně zabývá filozoficko-vědní obor **mereologie** (= studium meromonie).
- V UFO jsou diskutovány různé pohledy na problematiku Celek-Část (Ground Mereology, Minimum Mereology, Weak Supplementation, Extensional Mereology) vyznačující se různými specifickými vlastnostmi. Jejich detailní studium je však mimo rozsah tohoto kurzu.

# Různé typy vztahů Celek-Část

- Řada metodik a notací včetně UML se omezuje na nepřesné modelování např. "síly vazby":
  - Motor je nedílnou součástí auta. (**kompozice** = "silnější")




  - Pasažéři existují i bez auta. (**agregace** = "slabší")




- Pouze takovéto rozlišení je však zjednodušující a nejednoznačné pro ontologické modelování. Je třeba se zabývat zejména **oběma směry** (Část $\leftrightarrow$ Celek) a rozlišovat přesněji **"sílu" vzájemné povinnosti**.

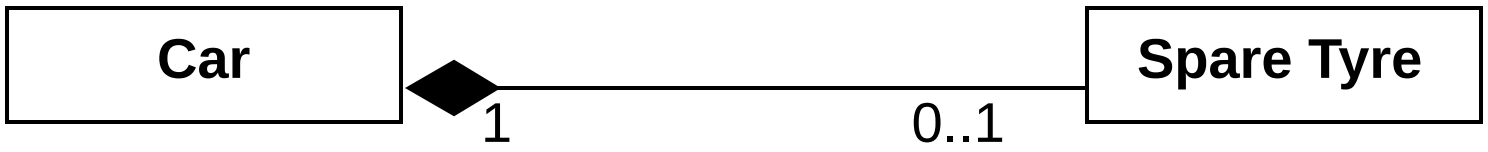
# Celek a Část

*„ It is as **impossible** to know the parts without knowing the whole as to know the whole without knowing the particular parts. “*

(Blaise Pascal)

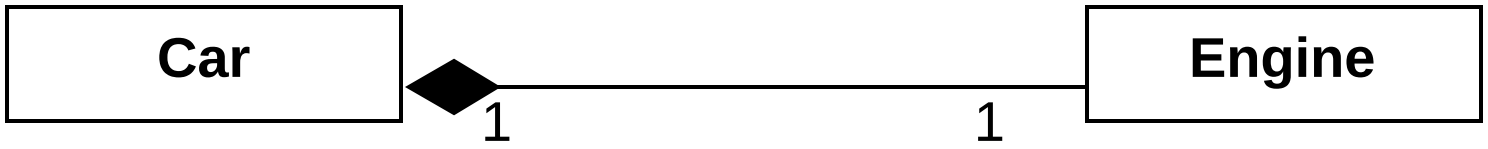
# Povinnost z hlediska Celku: Volitelná Část

- **Volitelné Části** jsou Části, které Celek může postrádat bez vlivu na jeho *klasifikaci či identitu*.



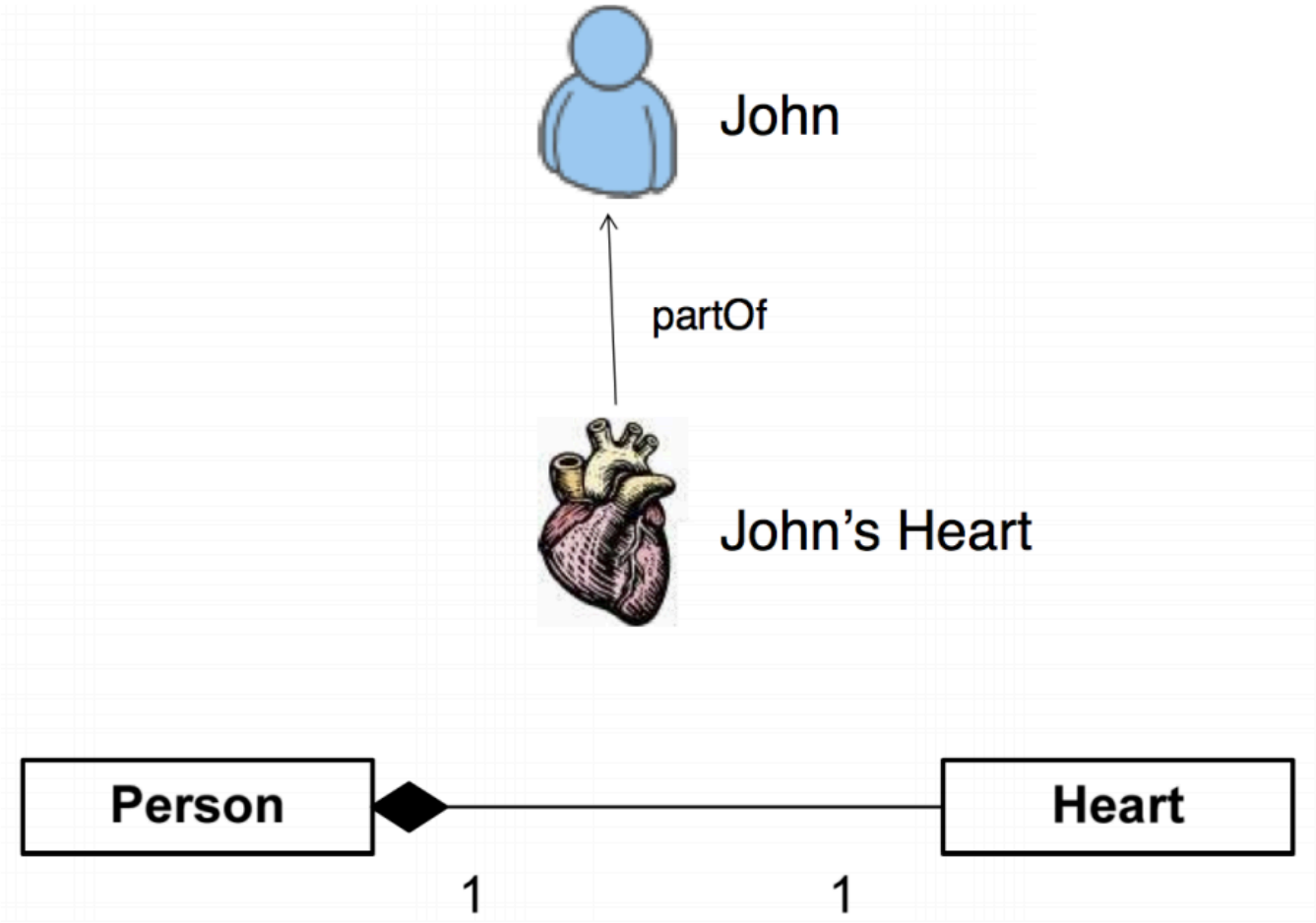
# Povinnost z hlediska Celku: Povinná Část

- **Povinné Části** jsou Části, které Celek naopak nemůže postrádat.

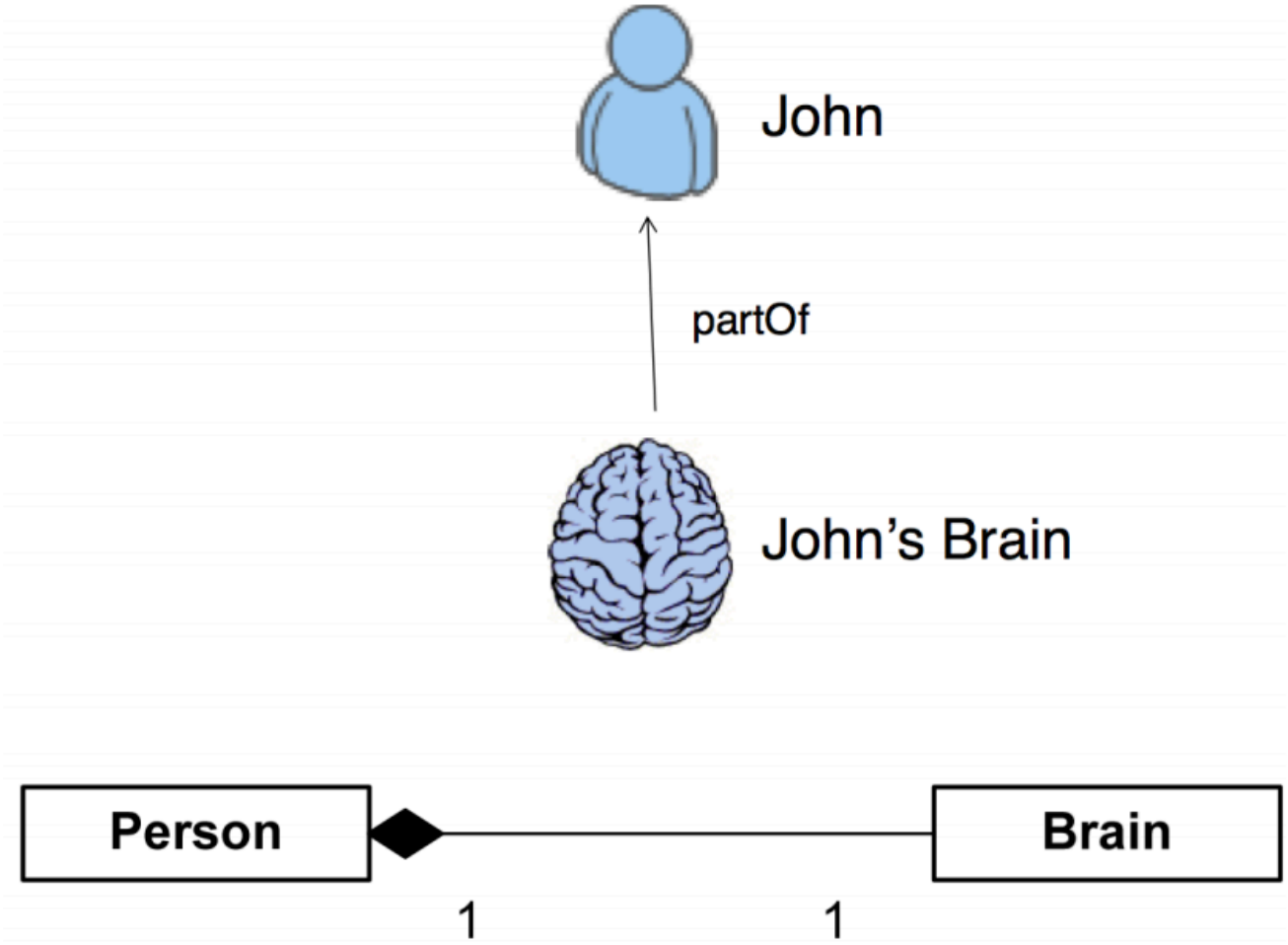



- Řada notací (včetně UML) se s tímto spokojí, avšak toto nestačí:

# Povinnost z hlediska Celku: Povinná Část



# Povinnost z hlediska Celku: Povinná Část



# Povinnost z hlediska Celku: Povinná Část

- Člověk musí mít srdce a musí mít mozek (bez nich nemůže žít).
- Srdce však lze transplantovat, tj. můžeme jej vyměnit za jinou instanci srdce bez vlivu na identitu člověka.
- Mozek oproti tomu nelze vyměnit za jiný kus; i pokud by to bylo technicky možné, znamenalo by to změnu identity člověka.
- Podobný rozdíl je i u vztahu Automobil-Motor a Automobil-Karoserie.
- Rozdíl spočívá v závislosti na *typu* vs závislosti na *konkrétní instanci*.
- Tj. *Člověk musí mít **nějaké** srdce* vs *Člověk musí mít **určitý** mozek*.

Tento rozdíl postihneme formálně pomocí pojmů *Generická závislost* a *Existenční závislost*:

# Generická závislost (Generic Dependence)

- Instance $y$ je genericky závislá (generically dependent) na typu $T$, pokud pro existenci $y$ je nutné, aby existovala i instance $T$.

$$GD(y,T) =_{def} \Box(\epsilon(y) \implies \exists x : T(x) \wedge \epsilon(x))$$

- $\epsilon(y)$ je unární predikát "existovat v doméně".

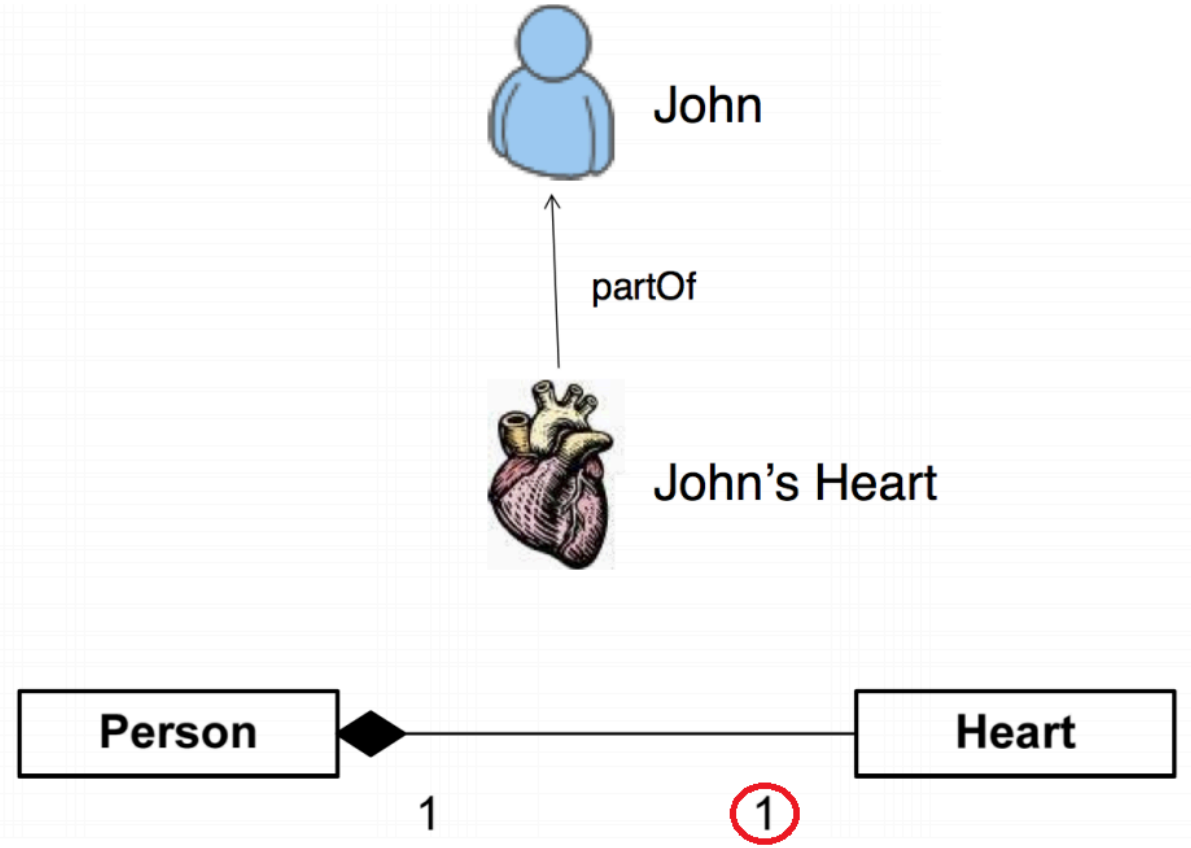
# Povinná část (Mandatory Parthood)

- Instance $x$ je povinná Část (mandatory part) jiné instance $y$, pokud $y$ je genericky závislá na typu $T$, jež $x$ instanciuje a instance $x$ je nezbytně Částí instance $y$:

$$MP(T, y) =_{def} \Box(\epsilon(y) \implies \exists x : T(x) \wedge (x \lhd y))$$

- $x \lhd y$ je binární predikát "$x$ je Částí $y$".

# Povinnost z hlediska Celku: Povinná Část




$$(\forall x : \text{Person}(x) \implies \Box(\epsilon(x) \implies \exists! y : \text{Heart}(y) \wedge (y \triangleleft x)))$$

# Existenční závislost (Existential Dependence)

- Instance $x$ je existenčně závislá (existentially dependent) na jiné instanci $y$, pokud existence instance $x$ vyžaduje existenci určité specifické instance $y$.

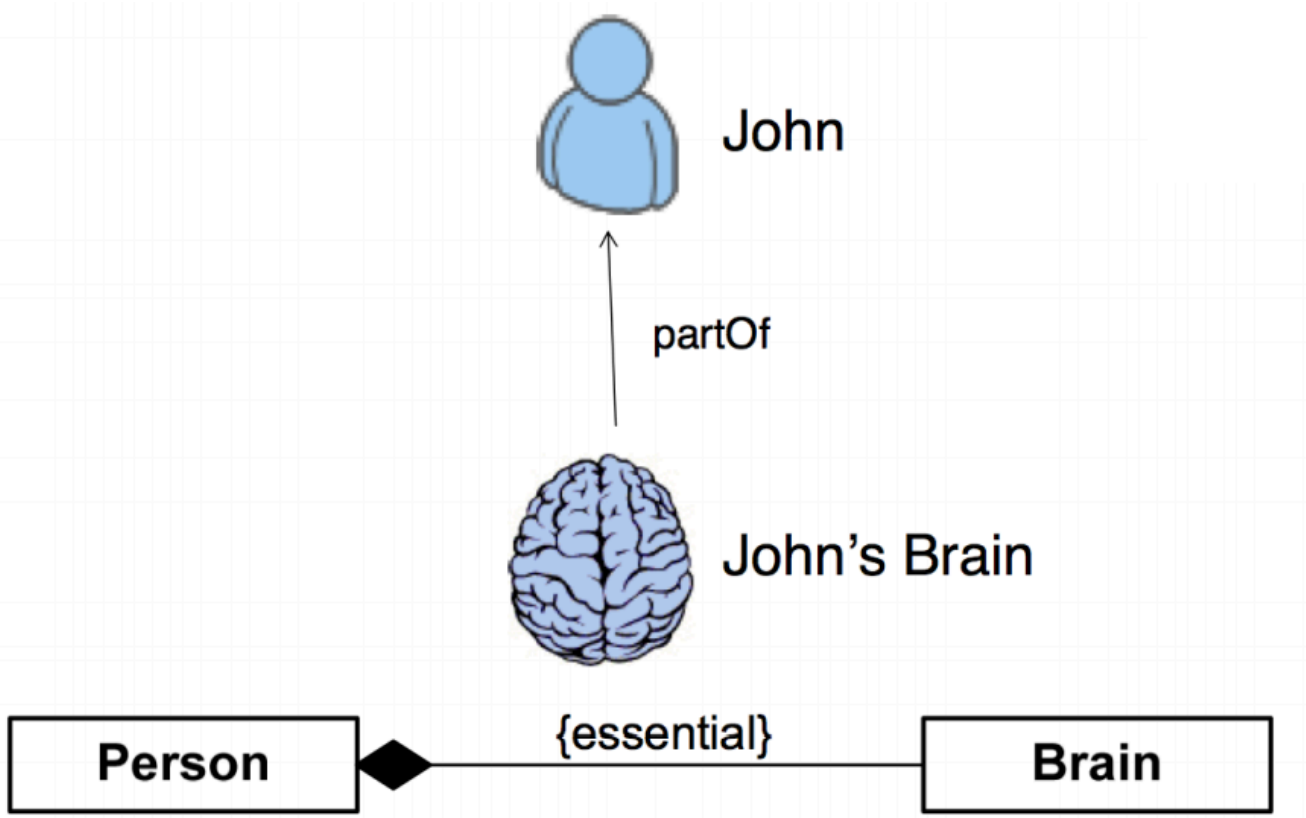
$$ed(x, y) =_{def} \Box(\epsilon(x) \implies \epsilon(y))$$

# Esenciální část (Essential Parthood)

- Instance $x$ je esenciální Část (essential part) jiné instance $y$, pokud $y$ je existenčně závislá na instanci $x$ a instance $x$ je nezbytně Částí instance $y$.

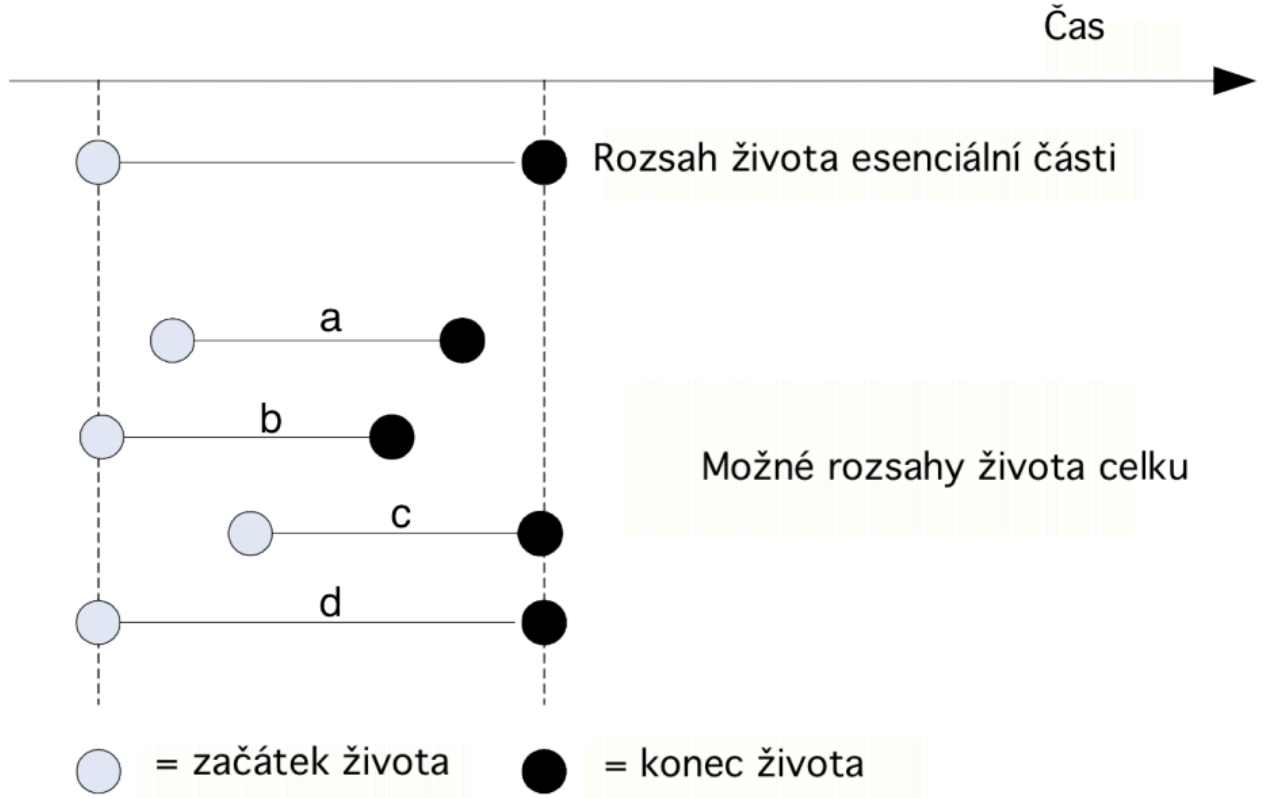
$$EP(x,y) =_{def} \Box(\epsilon(y) \implies (x \triangleleft y))$$

# Povinnost z hlediska Celku: Esenciální Část




$$\Box(\forall x : \text{Person}(x) \implies \exists! y : \text{Brain}(y) \wedge \Box(\epsilon(x) \implies (y \triangleleft x)))$$

# Povinnost z hlediska Celku: Esenciální Část




⚠️ Pouze rigidní typy mohou být použity jako esenciální části.

# Povinná vs. Esenciální Část

**Povinná Část**

$$(\forall x : \mathrm{Person}(x) \implies \Box(\epsilon(x) \implies \exists! y : \mathrm{Heart}(y) \wedge (y \triangleleft x)))$$

$\Box$ můžeme chápat jako iterátor přes množinu světů. Instance srdce se může měnit mezi jednotlivými světy (situacemi).

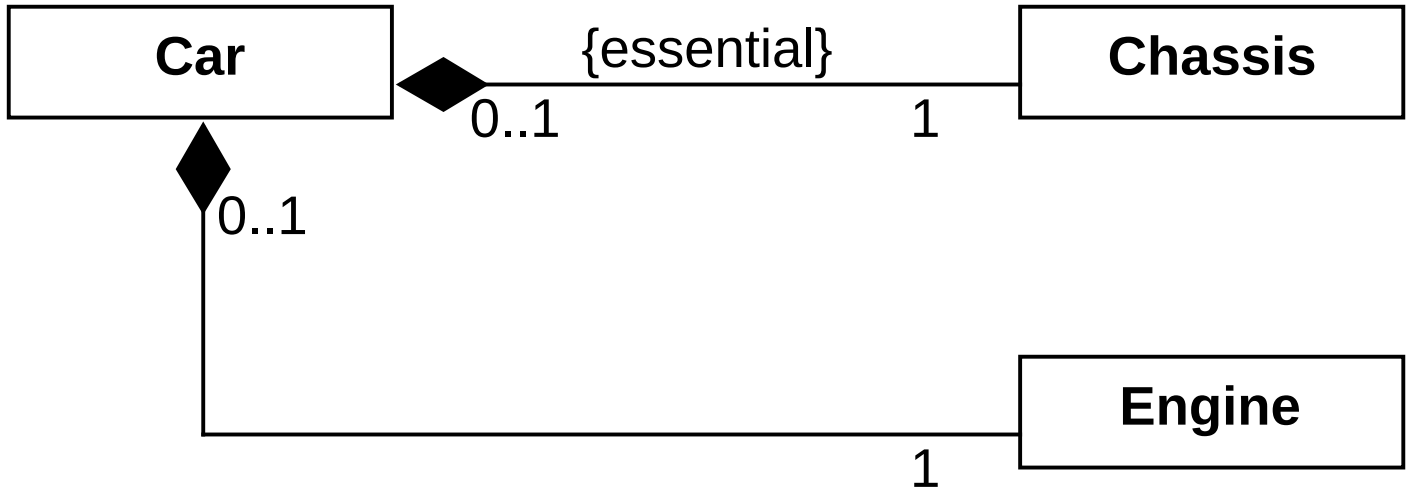
**Esenciální Část**

$$\Box(\forall x : \mathrm{Person}(x) \implies \exists! y : \mathrm{Brain}(y) \wedge \Box(\epsilon(x) \implies (y \triangleleft x)))$$

Instance mozku je vybrána než $\Box$ "začne iterovat" přes světy. Instance mozku je zafixována a nemůže se tak měnit.

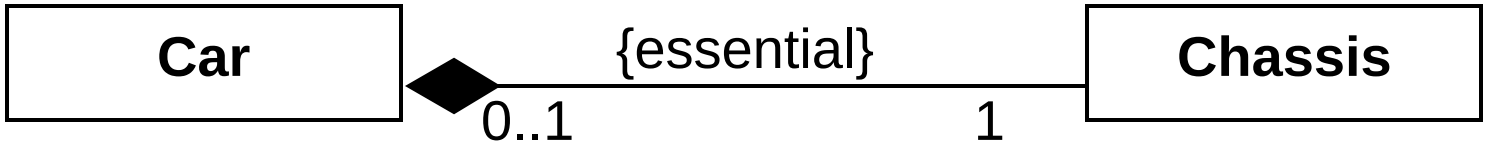
Pojem *povinná Část* je slabším požadavkem, a tudíž **esencialita implikuje povinnost**.

# Povinná vs. Esenciální Část



# Povinnost z hlediska Části: Nepovinný Celek

- Část může existovat bez Celku.

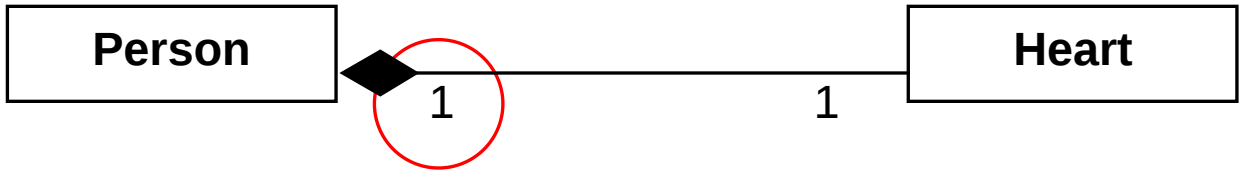



*Pro úplnost: zde se jedná o esenciální část s nepovinným celkem.*

# Povinnost z hlediska Části: Povinný Celek

- Instance $y$ je povinný Celek (mandatory whole) pro jinou instanci $x$, pokud $x$ je generický závislá na typu $T$, který $y$ instanciuje, a instance $x$ je nezbytně Částí instance typu $T$:

$$MW(T, x) =_{def} \Box(\epsilon(x) \implies (\exists y : T(y) \wedge (x \triangleleft y)))$$

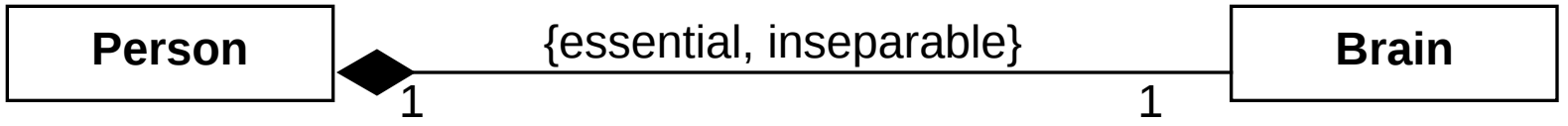



$$\Box(\forall x : \mathrm{Heart}(x) \implies \Box(\exists! y : \mathrm{Person}(y) \wedge (x \triangleleft y)))$$

# Povinnost z hlediska Části: Neoddělitelná Část

- Instance $x$ je neoddělitelná Část (inseparable part) jiné instance $y$, pokud $x$ je existenčně závislá na instanci $y$ a instance $x$ je nezbytně Částí instance $y$:

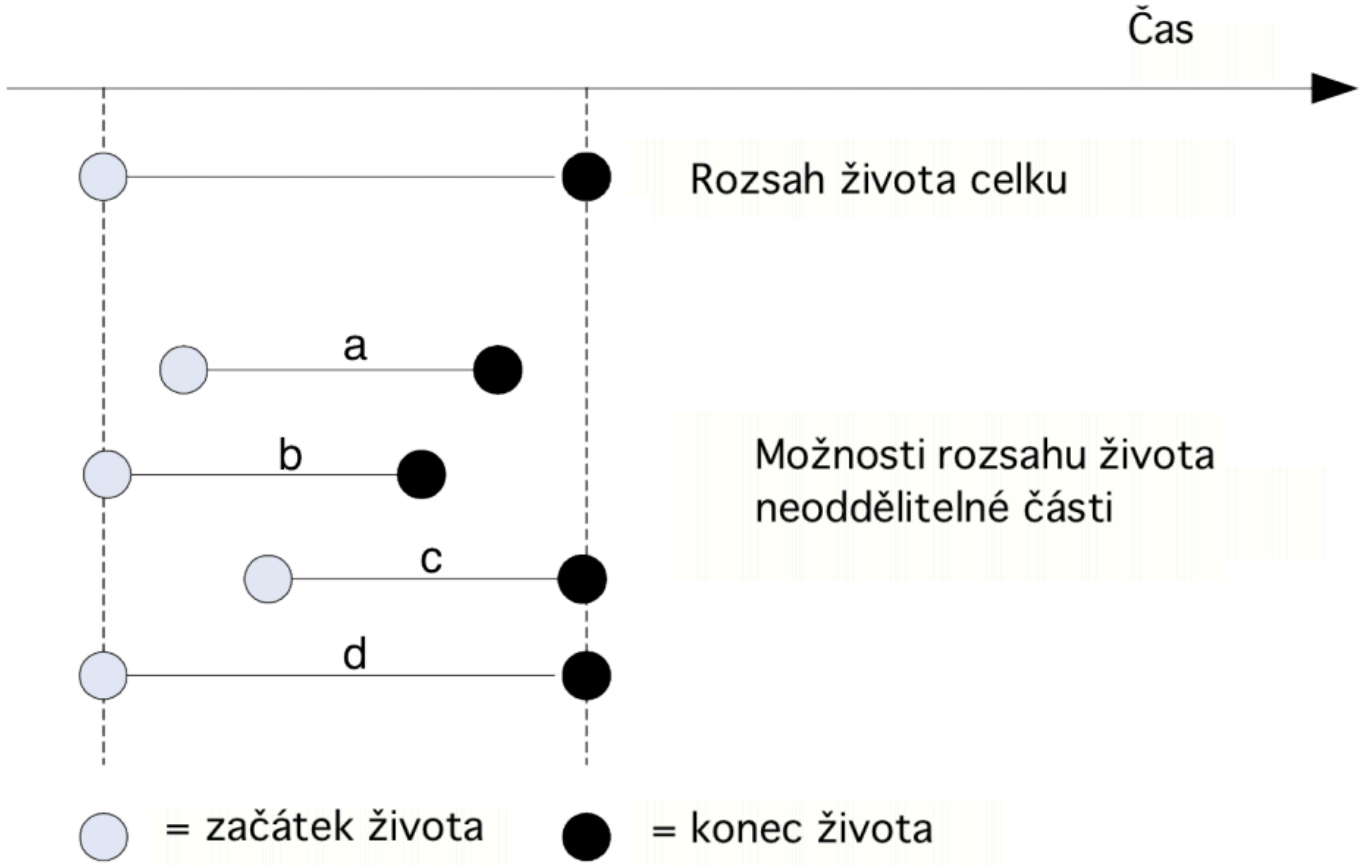
$$IP(x, y) =_{def} \Box(\epsilon(x) \implies (x \lhd y))$$



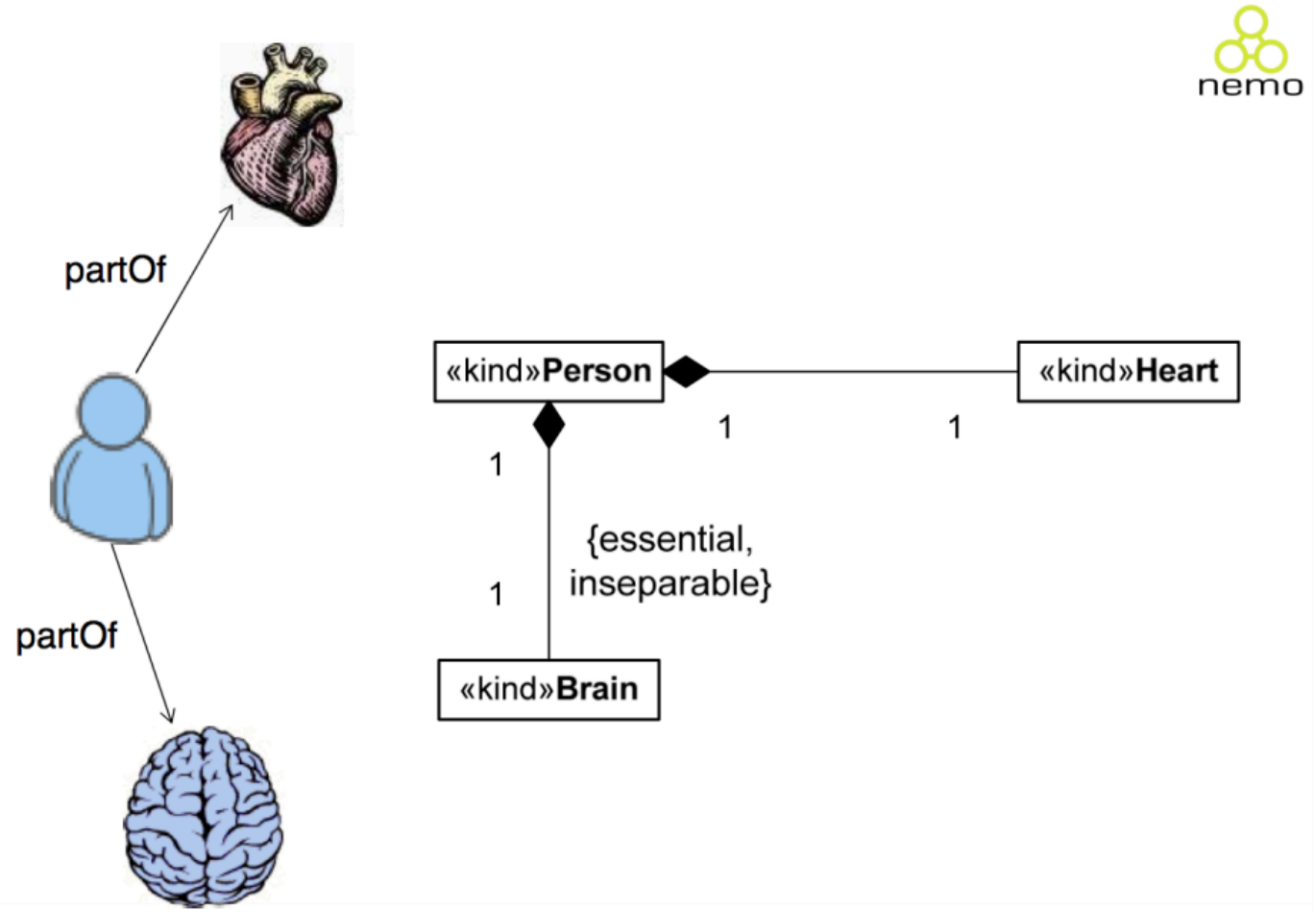

$$\Box(\forall x : \mathrm{Brain}(x) \implies (\exists! y : \mathrm{Person}(y) \wedge \Box(\epsilon(x) \implies (x \lhd y))))$$

⚠️ Neoddělitelnost implikuje povinnost Celku.

# Povinnost z hlediska Části: Neoddělitelná Část



# Neoddělitelná esenciální část



# Esencialita vs. neoddělitelnost

Jedná se o zcela nezávislé pojmy:

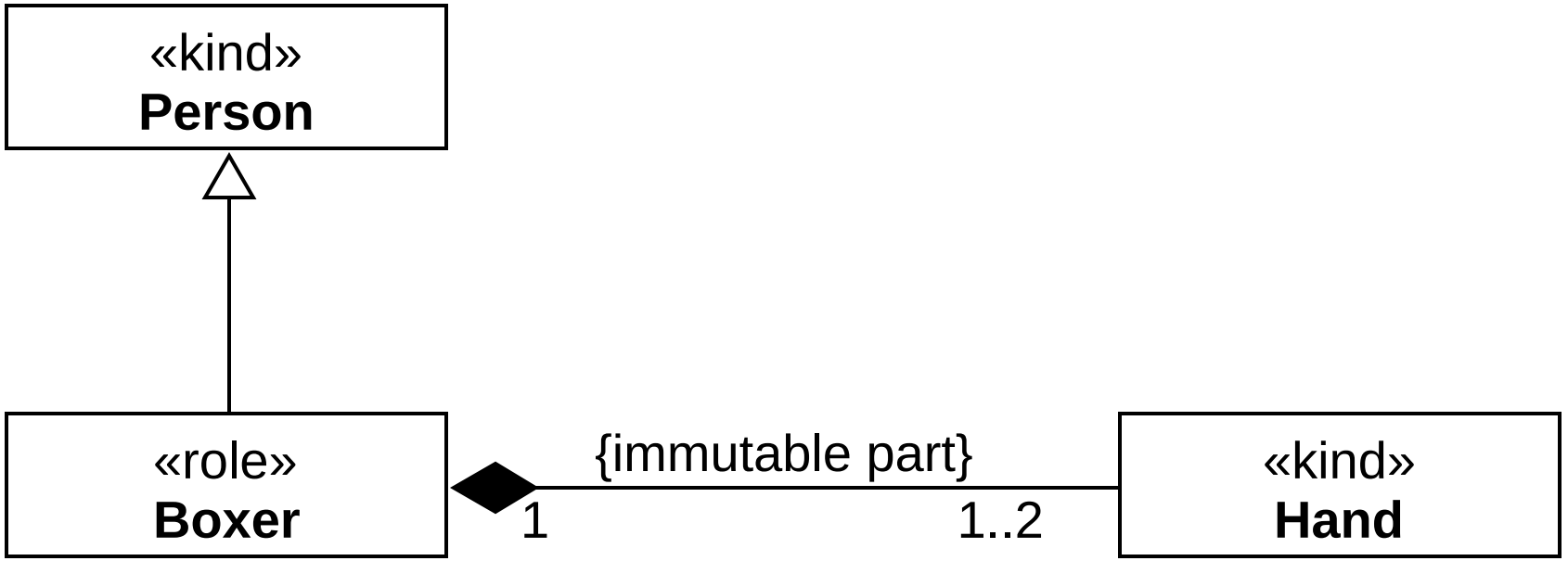
- **Esencialita neimplikuje neoddělitelnost**:
  - Např. publikace ”Sebrané dílo“ nějakého autora: Části jsou esenciální (všechny definují identitu tohoto díla), nicméně každá Část existovala i před vytvořením této publikace.
- **Neoddělitelnost neimplikuje esencialitu**:
  - Díra ve zdi je její neoddělitelná Část, ale nikoliv esenciální.



# Neměnitelná Část (Immutable Part)

- A co non-rigid typy? Říkali jsme, že je nelze použít pro essential/inseparable (došlo by k porušení definice)...
- Nicméně, analogicky k *essential* ale můžeme požadovat, aby když má rigidní celek nějakou podčást, ta ho musí doprovázet ve všech světech, kde existuje = **neměnitelná část (Immutable Part)**.

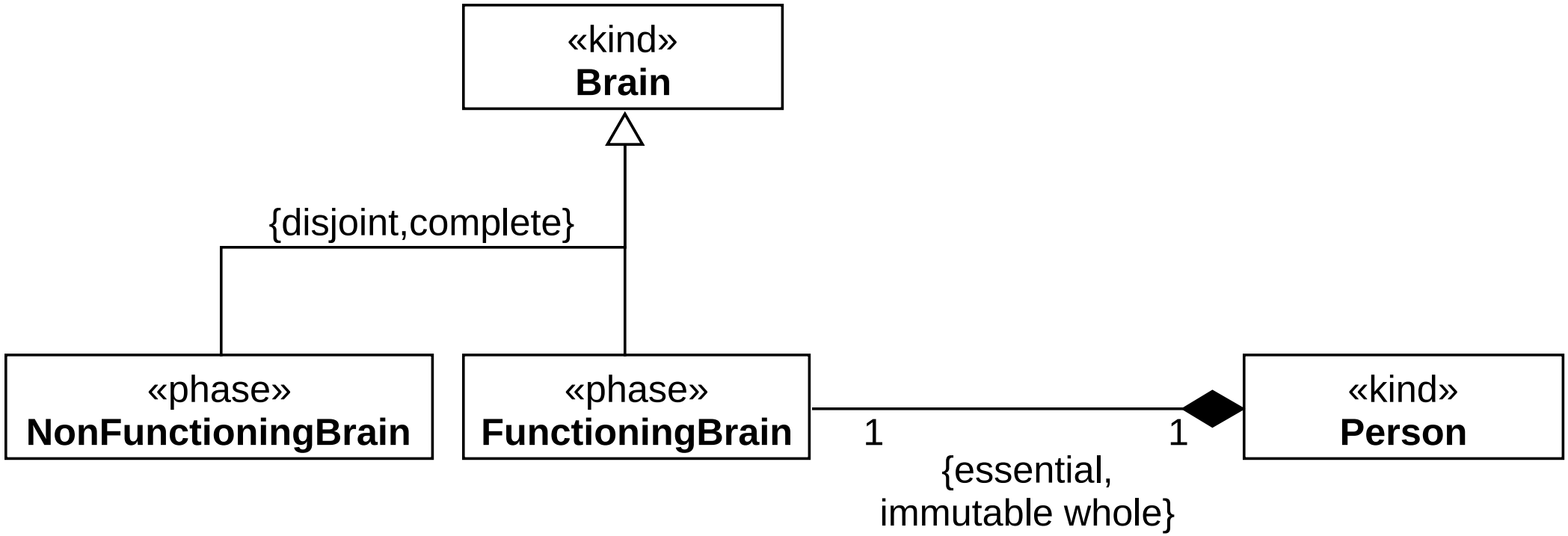
# Neměnitelná Část (Immutable Part)




$$\Box(\forall x : \mathrm{Boxer}(x) \implies \exists y : \mathrm{y} \wedge \Box(\epsilon(x) \wedge \mathrm{Boxer}(x) \implies (y \triangleleft x)))$$

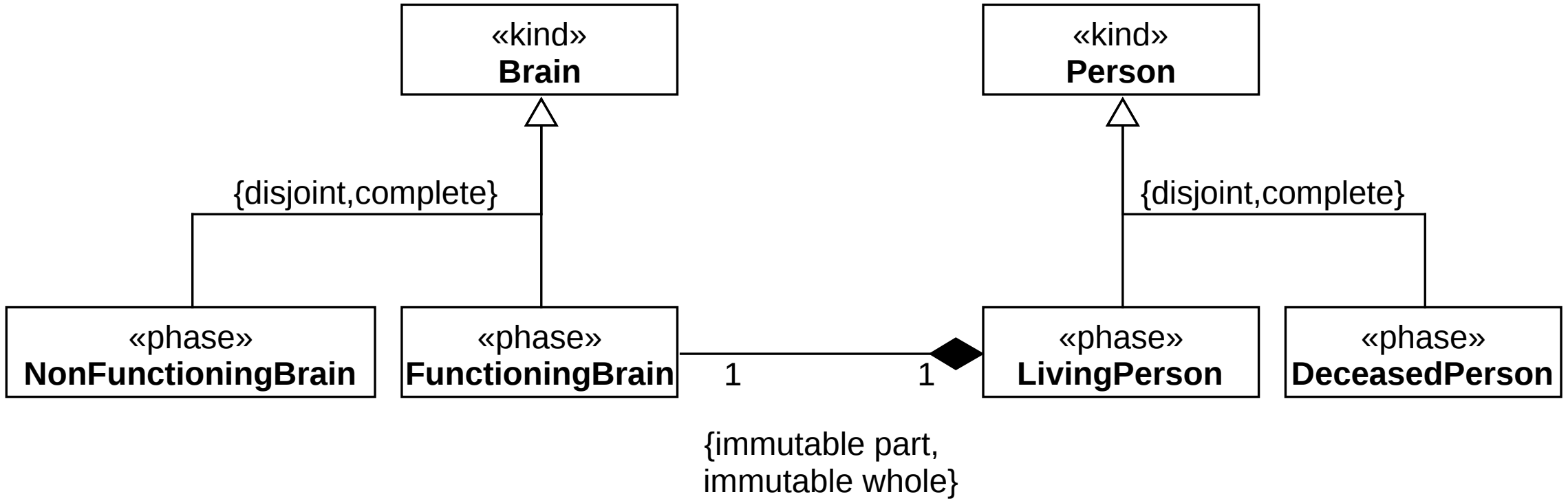
# Neměnitelný Celek (Immutable Whole)

- Směrem od Části k Celku u Non-Rigid typů můžeme hovořit o **neměnitelném Celku (Immutable Whole)** jakožto paralele k *inseparable*:



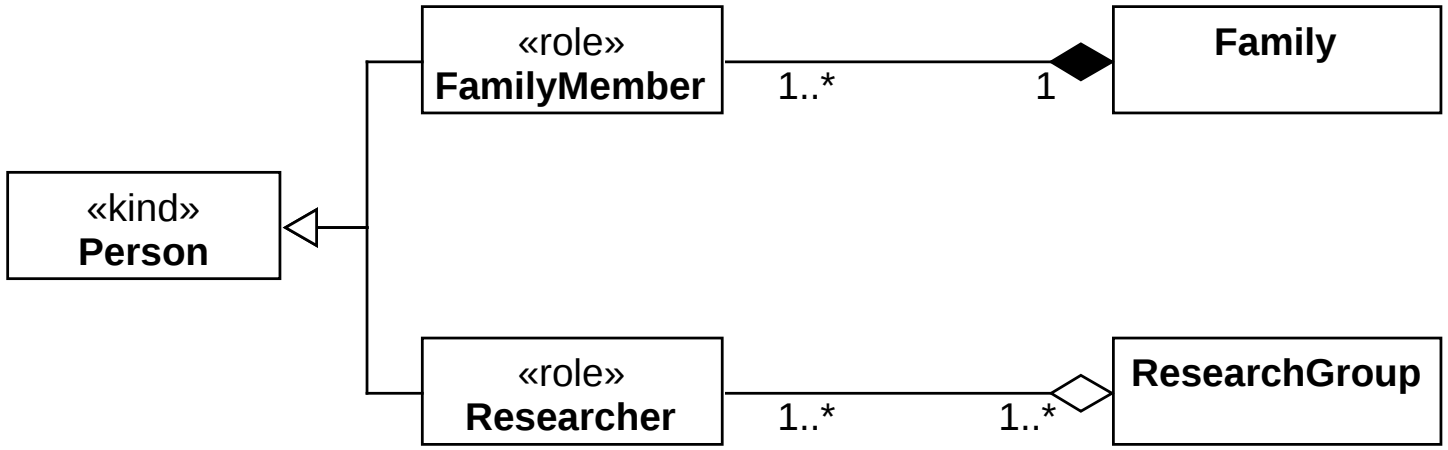
# Neměnitelný Celek a neměnitelná Část

- Opět se jedná o nezávislé pojmy.



# Sdílitelnost (shareability) - "plný" a "prázdný" diamant

- Vágní pojmy *kompozice* a *agregace* z UML jsou v OntoUML nahrazeny přesnějšími.
- Nicméně OntoUML musí být syntakticky kompatibilní s UML, tedy se musí mít význam i pro "plný/prázdný diamant": použito pro vizualizaci *sdílitelnosti*, která je ovšem odvoditelná z multiplicity:
  - ♦ = `{isShareable = false}`, nesdílitelná část, multiplicita max. 1
  - ◇ = `{isShareable = true}`, sdílitelná část, multiplicita max. > 1



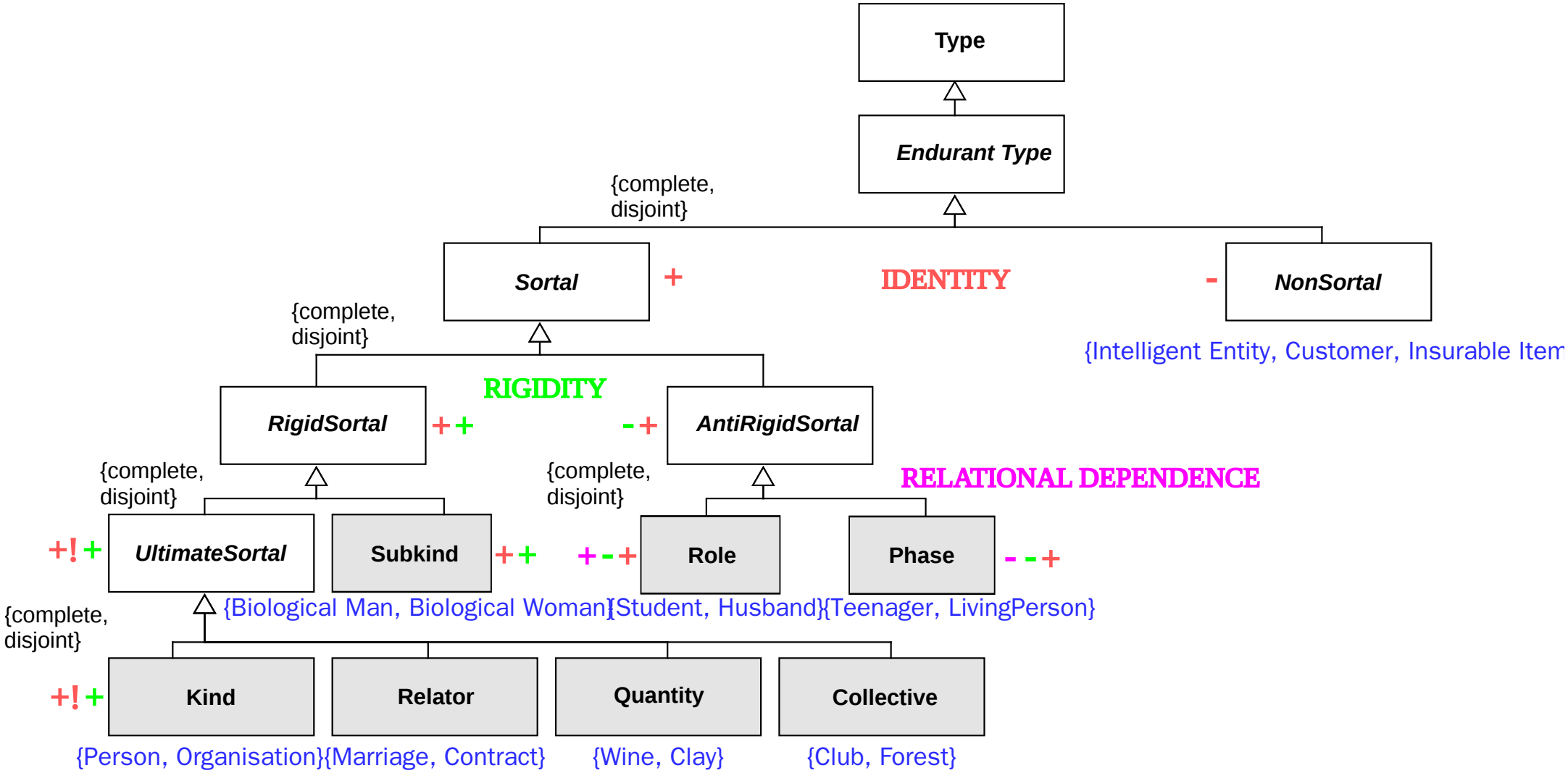
# Shrnutí vztahů Celek-Část

- Vztahy Celek-Část jsou zpřesněním asociace *obsahuje (has)*.
- Představují důležitý mechanismus vnímání naší reality, jelikož umožňují systém vnímat na různých úrovních (od obecného celku k detailu částí).
- Rozlišujeme **různé druhy povinnosti**:
  - Směrem od Celku k Částem (optional, mandatory, essential).
  - Směrem od Části k Celku (optional, mandatory, inseparable part)
- U non-rigidních typů můžeme vyžadovat **neměnnost** části (immutable part) a celku (immutable whole) vzhledem k různým světům.
- Multiplicita 1..* implikuje *možnost sdílet* část vícero celky (shareability).
- Multiplicita 1 implikuje *nesdílitelnost* části vícero celky (non-shareability).

# Typy agregací Celku a Částí

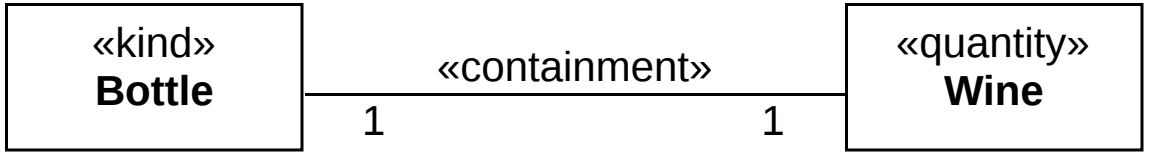
- Dosud jsme uvažovali `Kind` (či podtypy) jako agregující typ (celek).
- Studie kognitivních věd a lingvistiky však ukazují, že vnímáme vícero různých agregujících typů s různými vlastnostmi a vazeb Celek-Část:
  - Kvantita ( `Quantity` )
    - `containment` : víno-láhev, malta-kyblík, jíl-socha, ...
    - `subQuantityOf` : alkohol-víno, písek-malta, křemík-jíl, ...
  - Kolektiv ( `Collective` )
    - `memberOf` : strom-les, student-paralelka, pták-hejno, ...
    - `subCollectionOf` : severní cíp lesa-les, mladí ptáci-hejno, ...
  - Funkční celek (Functional Entity)
    - `componentOf` : srdce-oběhový systém, ředitel-firma, motor-automobil, ...

# Sortální typy

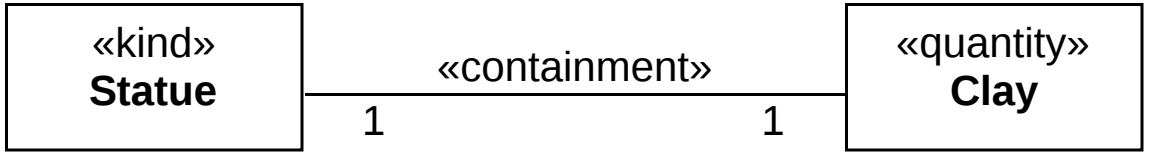


# Kvantita («Quantity»)

- Kvantita je typ celku, pro který je charakteristické, že je **složen s částí, které jsou stejného typu jako on sám**.
- Jedná se typicky o kapalné, vazké a sypké materiály, které zaujímají tvar nádoby, ve které jsou (víno, voda, med, písek, ...):



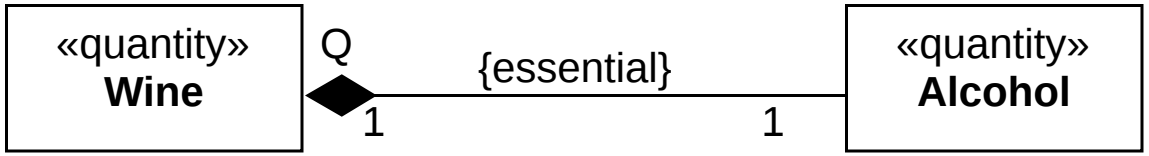

- Dále se jedná o materiály, ze kterých jsou vyrobeny hmotné objekty (jíl, plast, dřevo, ...):



# Kvantita («Quantity»)

- Kvantita je agregovaný celek, který je vždy **maximální spojitý objekt**:
  - *spojitý* = každá část je v prostoru spojena s každou další částí a plynule v ní přechází,
  - *maximální* = neboli není podčástí nějaké větší části stejného typu.
- Kvantita je tedy **vymezena topologicky**, tj. zabírá určitý kontinuální kompaktní prostor.
- Zde modelujeme **množství látky**, např. vodu či jíl. Přednáška je událost; peněžní částka není automaticky `Quantity`.

# Relace SubQuantityOf




- Relace `SubQuantityOf` mezi dvěma kvantitami (resp. podtypy) je vyznačena **Q**.
  - Kvalifikátor by měl být nejlépe uvnitř diamantu, ale ne každý nástroj to umí.
- Jelikož všechny Podkvantity jsou též maximální, kardinalita na straně Části musí být vždy **právě 1**.
- Vždy se jedná o **esenciální část**: celek je existenčně závislý na instanci svého složení (zkuste vydestilovat alkohol z vína a nahradit ho jiným...).
- Relace je **anti-symetrická** (platí pro všechny relace celek-část).
- Relace je **tranzitivní** a **irreflexivní** (vyplývá z maximality).

# Kolektiv («Collective»)

- Kolektiv je oproti Kvantitě složen z částí, které **nejsou stejného typu jako on sám** (strom není les).
- Kolektiv **není nekonečně dělitelný**, nýbrž dojdeme nakonec k atomické jednotce mající identitu = **Prvek (Element, Entity)**.
- **Všechny Prvky hrají v kolektivu stejnou roli** (= charakteristická odlišnost od Funkčního Celku, viz dále).
- V informatice se typicky pro konceptualizaci kolektivů používá konstrukt matematické množiny.
- Je ovšem rozdíl mezi "množinou" v kognitivním pojetí a množinou matematickou.

# Celek a Část

*„ Mathematics may be defined as the subject in which we never know what we are talking about, nor whether what we are saying is true. “*

(Bertrand Russell)

# Integrální Celek

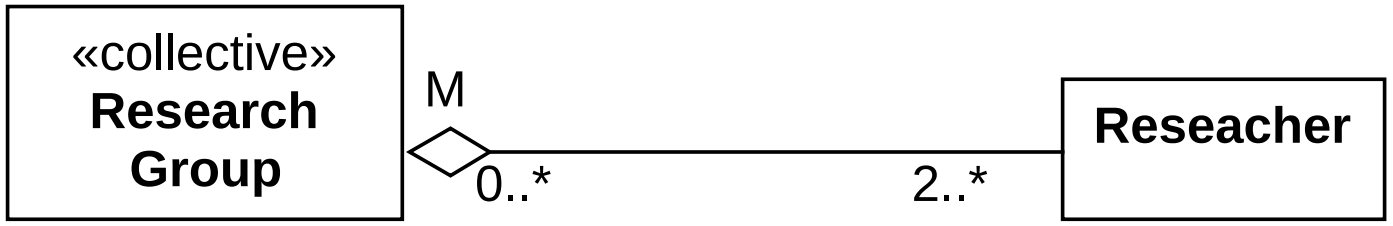
- Matematická množina znamená určitý soubor přesně vymezených objektů, které buď do množiny patří nebo nepatří.
  - Příklad: $\{\text{klá vesnice}, \text{strom}, \text{Praha}, \text{Jupiter}, \text{slabost}\}$
- Oproti tomu v našem chápání vždy vytváříme množiny na základě určitého "klíče", podobně jako intenzivní vymezení matematické množiny.
- Ovšem zde místo vzorečku figuruje ontologické vymezení "co tyto prvky sdružilo": tzv. **charakteristická (sjednocující) relace** .
  - Příklad: třída studentů (sjednocující relace: *společně studovat ve třídě X*)
- Pojem **Integralní Celek (Integral Whole)** je poté "kognitivní verzí matematické množiny".
- Vyhneme se tak i obtížně ontologicky interpretovatelným konstruktům jako např. množiny množin typu $\{a\}, \{\{a\}\}, \{\{\{a\}\}\}, \ldots$ či množina s prázdným prvkem, apod.

# Kolektiv («Collective»)

- Kolektiv tedy není vymezen topologicky jako kvantita, ale je **vymezen sjednocující relací** – prvky se mohou topologicky nacházet libovolně (*můj Kolektiv kamarádů může být rozprostřen po celém světě*).
- Kolektiv je vždy **maximální entita přes sjednocující relaci** = analogie s maximálností *Quantity*.
- Sjednocující relace není v OntoUML modelu však explicitně vyjádřena.

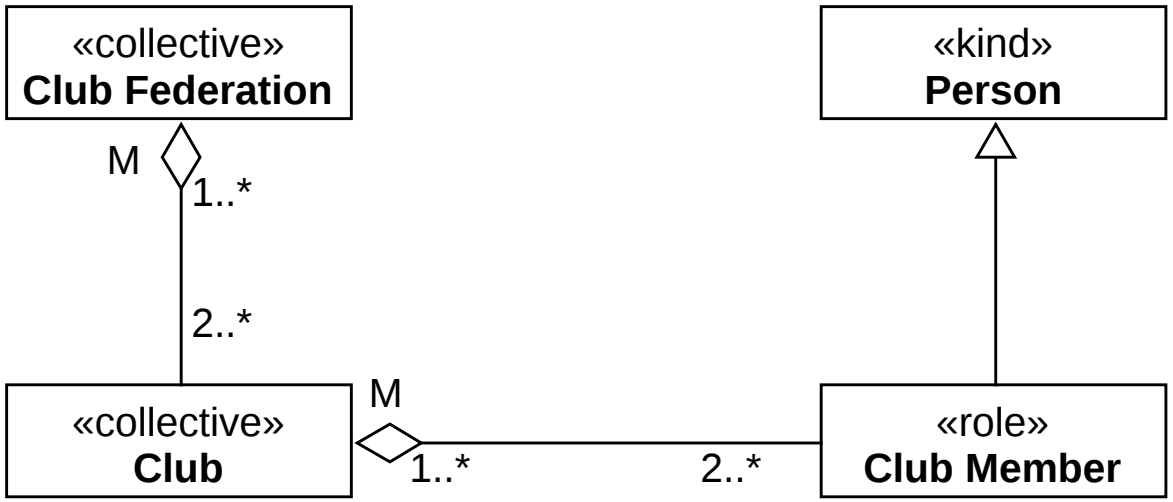
# Relace MemberOf

- Relace **mezi Prvkem a Kolektivem**.
- Kolektiv musí mít alespoň 2 členy, aby mohla existovat sjednocující relace; jinak to z podstaty není kolektiv = další rozdíl oproti matematické množině.
- Často zde Prvky nazýváme též **Atomická entita**, nebo zkráceně **Atom**, abychom zdůraznili nedělitelnost Prvku.



# Relace MemberOf

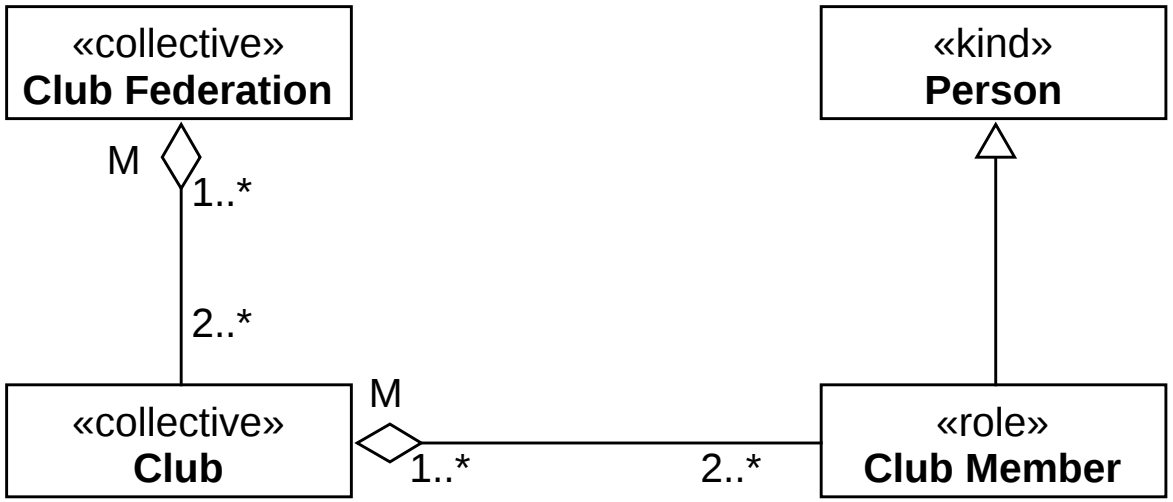
- Pozor ale na to, že se jedná o **nedělitelnost z hlediska Prvku vůči Kolektivu**, tedy neříká se nic o dělitelnosti Prvku samotného:




- Klub je z hlediska Federace atomická jednotka, ale samotný Klub má své členy.

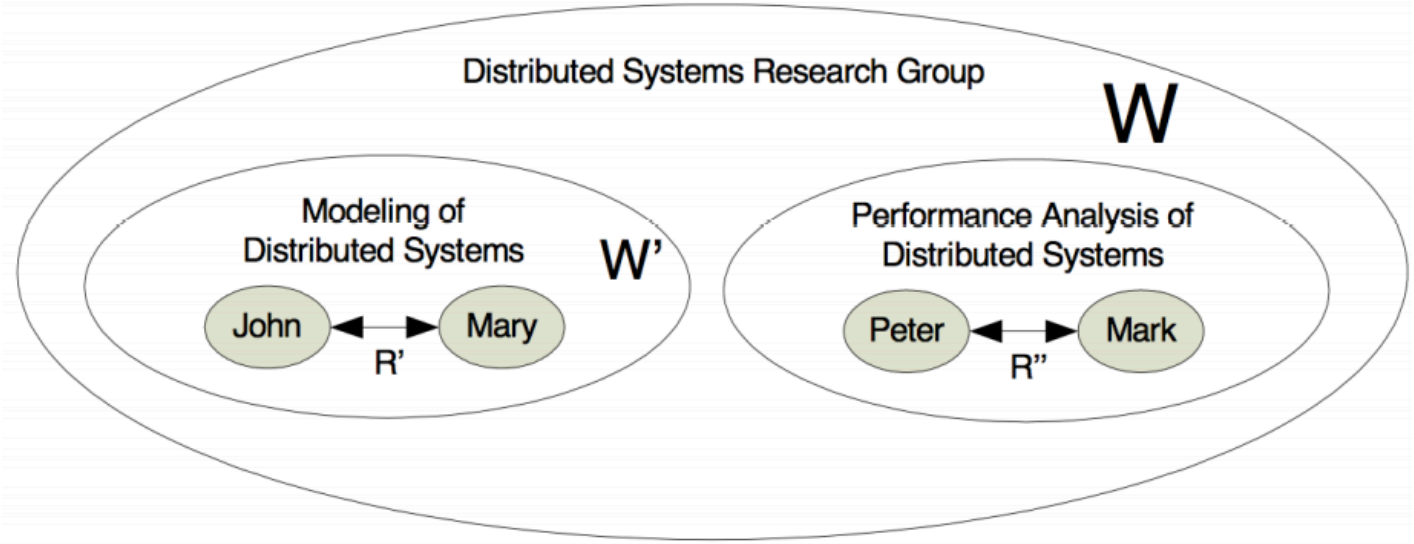
# Relace MemberOf

- Relace `MemberOf` je **irreflexivní**, **anti-symetrická** a **NEtranzitivní** (rozdíl oproti relaci `SubCollectionOf` dále):



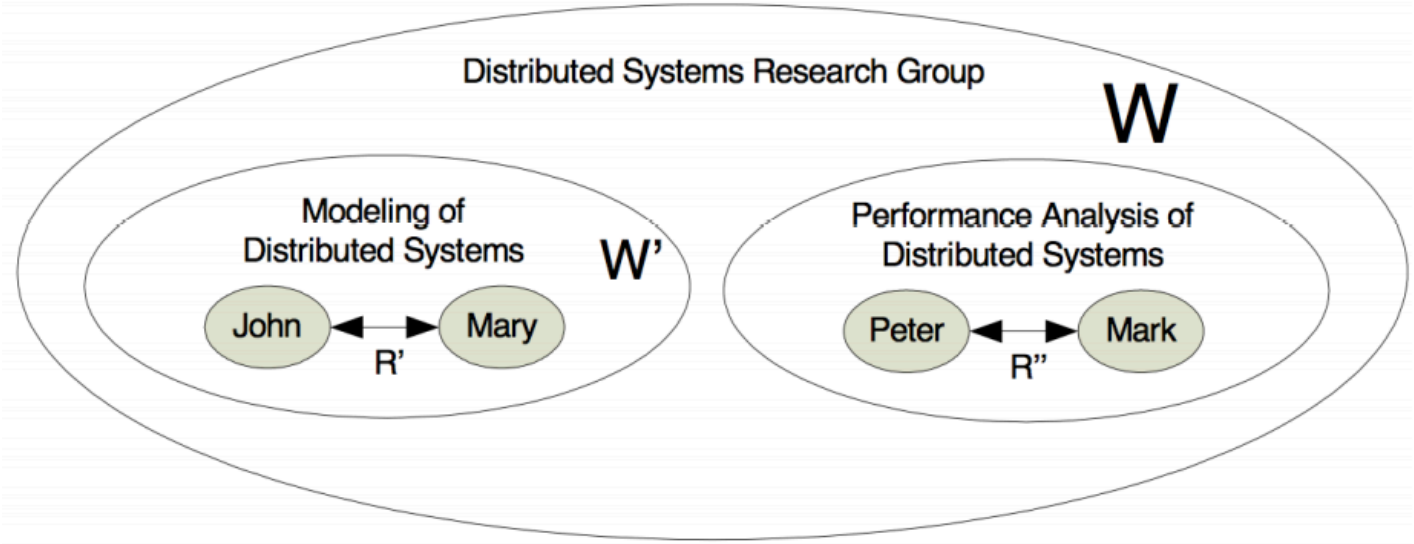

- Jsem členem klubu (Kolektiv) a můj klub je členem Federace (Kolektiv). Z toho ale nevyplývá, že jsem členem Federace, protože členové Federace mohou být pouze kluby, nikoliv jednotlivci.

# Relace SubCollectionOf



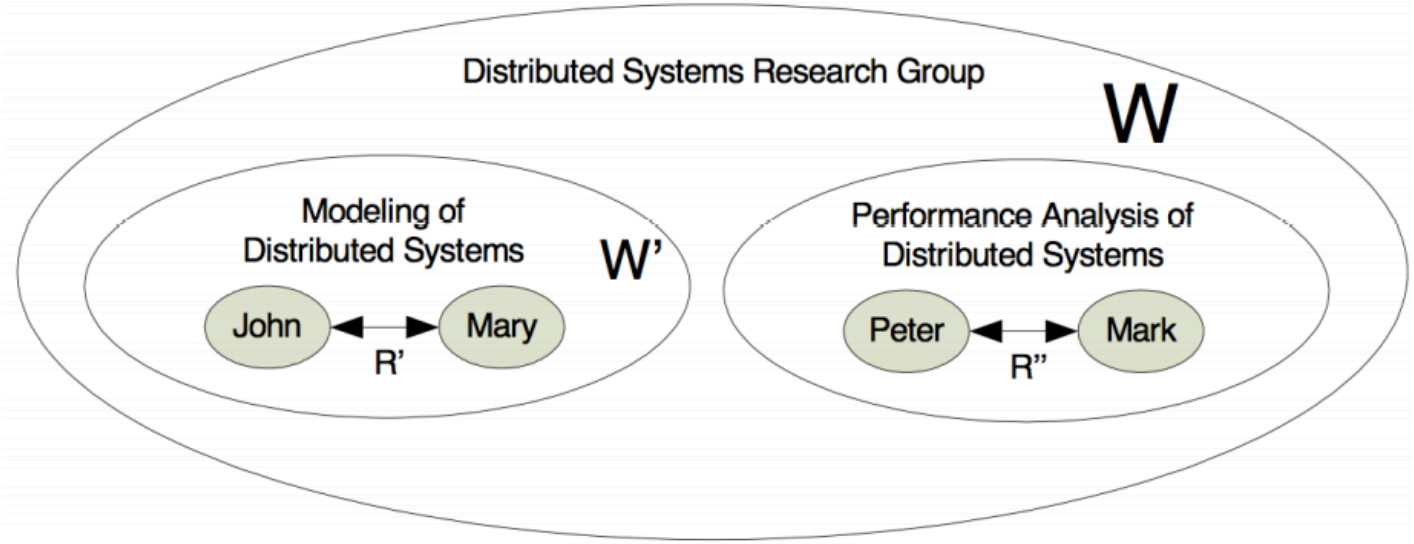

- Relace mezi **dvěma plurálními entitami** (oproti relaci `MemberOf`).
- Sjednocující relace $R'$ Podkolektivu $W'$ je specializací sjednocující relace $R$ Kolektivu $W$.
- Podkolektiv $W'$ Kolektivu $W$ je Kolektiv takový, že všechny Prvky Kolektivu $W'$ jsou současně Prvky Kolektivu $W$ = **transitivní relace**.

# Relace SubCollectionOf




- Jelikož Kolektiv je vždy maximální entita přes sjednocující relaci:
  - Kolektiv nemůže mít Podkolektiv stejného typu (tj. přes stejnou sjednocující relaci) = **irreflexivní relace**
  - Kolektiv může mít max. 1 Podkolektiv určitého typu.
- Stejně jako všechny celek-část relace je relace navíc **anti-symetrická**.

# Relace SubCollectionOf

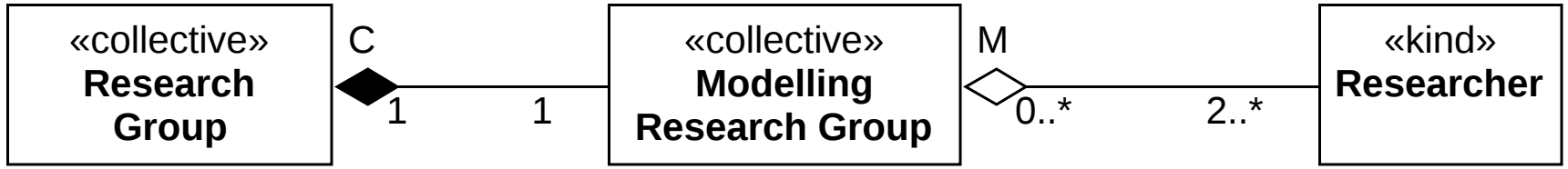



- Ze `SubCollectionOf` samotného **neplyne** `{inseparable}`; to vyžaduje doménově danou závislost na tomto konkrétním celku.
- Podkolektiv může být sdílený: táž skupina sourozenců může patřit současně do rodiny i sportovního týmu.

OntoUML.org: SubCollectionOf

# Relace MemberOf + SubCollectionOf

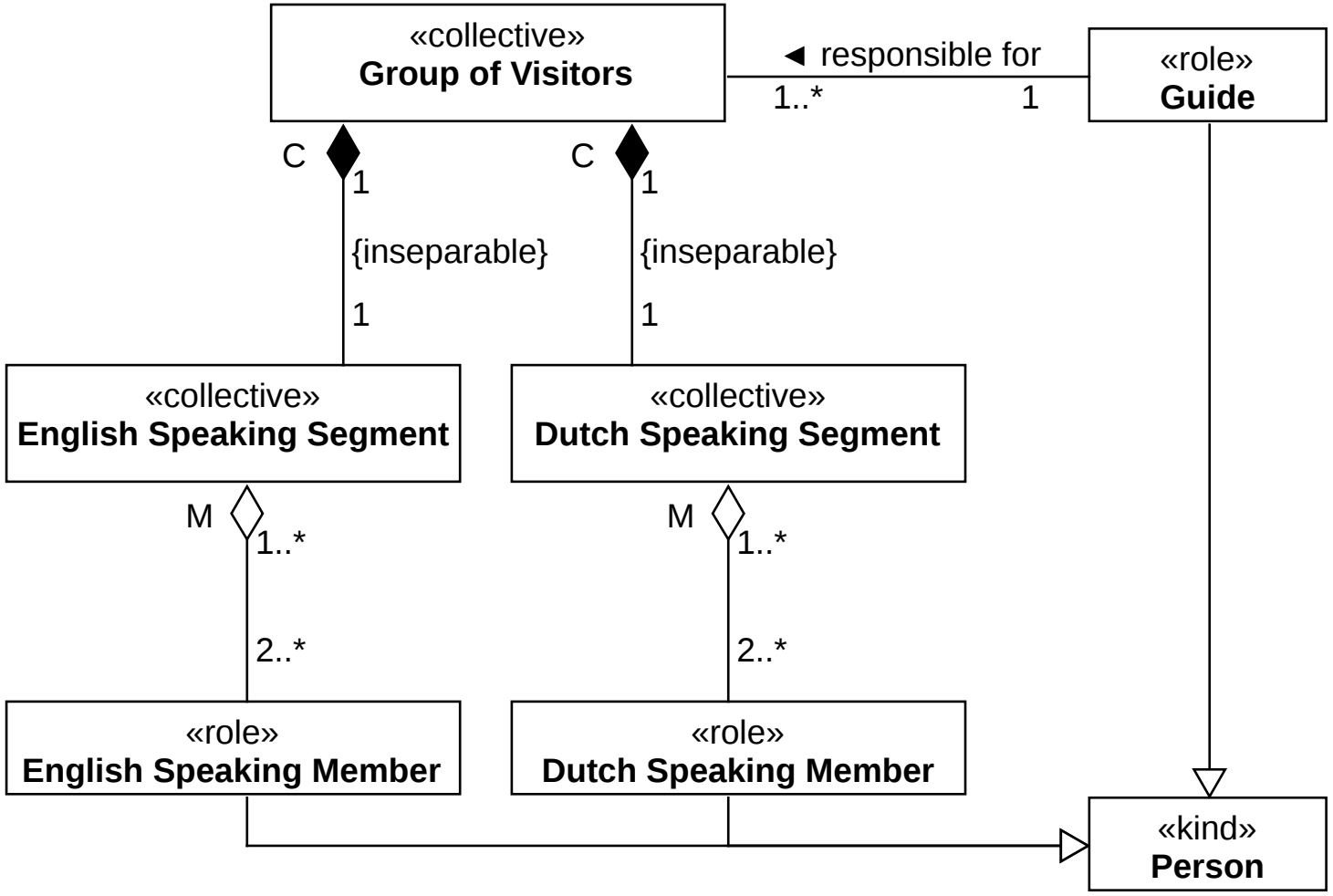
- Kombinace relace `MemberOf` a `SubCollectionOf` **zachovává tranzitivitu**: Pokud je Prvek členem Podkolektivu, je i členem Kolektivu.
  - Pokud je *Výzkumník* členem Podkolektivu *Modelování*, je i členem Kolektivu *Výzkumná skupina*.



# Význam tranzitivity

- Tranzitivita zajišťuje např. propagaci práv a závazků (člen fakulty získává tranzitivně práva a závazky člena univerzity).
- Jedná se o podobný princip jako u dědičnosti, přes tranzitivitu se ovšem nepropagují automaticky všechny atributy a vztahy.

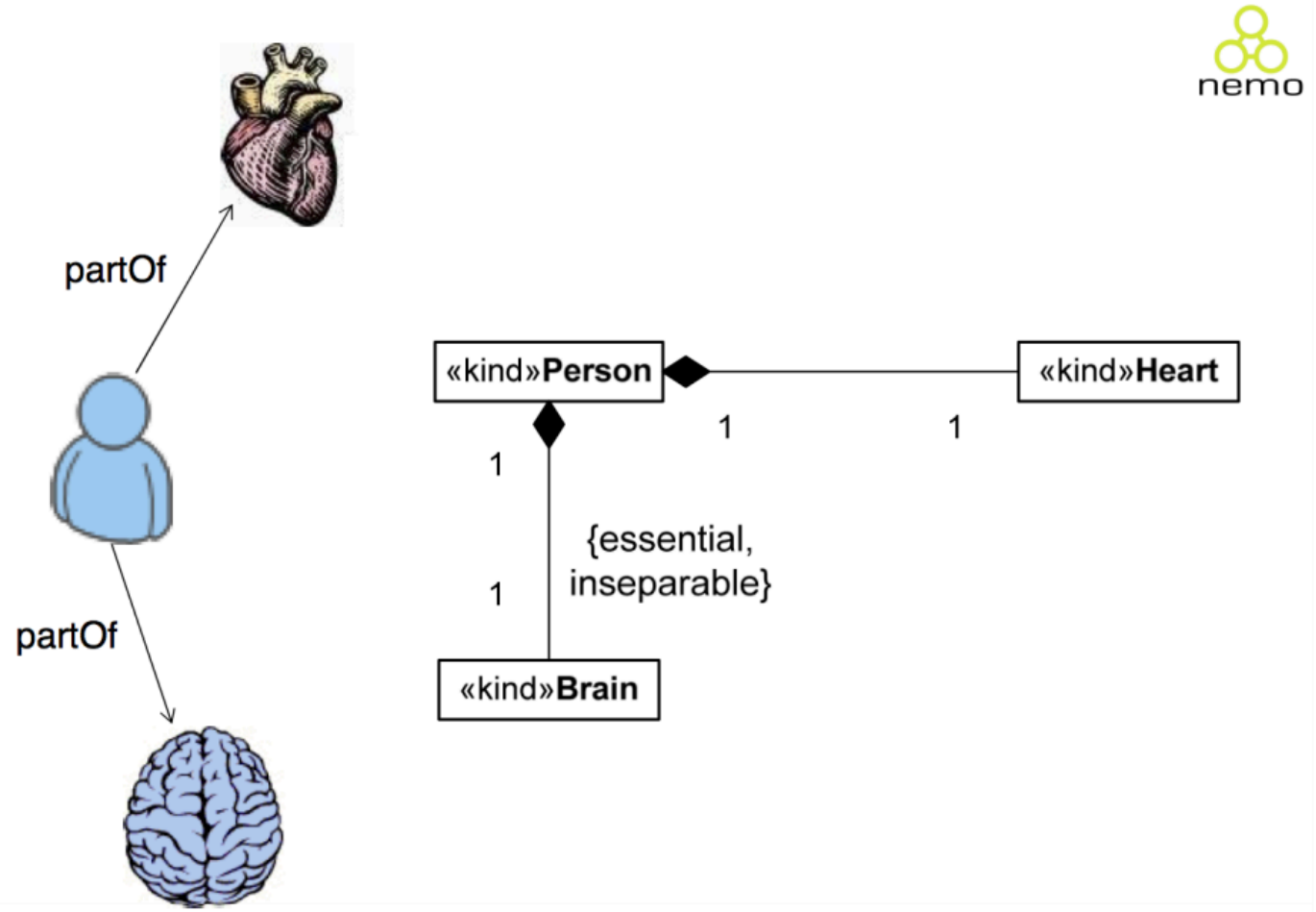
# Kolektiv - příklad



# Funkční celek

- Nejběžnější agregovaný typ.
- Funkční celek se skládá z částí, které mají **různé úlohy/funkce v rámci celku**. Tím se liší od Kolektivu, kde všechny Prvky jsou ekvivalentní.
- Funkční celky mohou mít volitelné, povinné a esenciální části.
- Relace `ComponentOf`:
  - Váže části k Funkčnímu celku.
  - Je **irreflexivní** a **anti-symetrická**.
  - **Není obecně tranzitivní** (v některých případech je, v jiných není).

# Funkční celek



# Shrnutí typů agregací

- Rozlišujeme 3 základní typy agregací: **Kvantita**, **Kolektiv**, **Funkční celek**.
- Tyto typy se liší svými vlastnostmi vzhledem k definici agregace, tranzitivitě, esencialitě prvku.
- `Quantity` a `Collective` jsou stejně jako `Kind` a `Relator` rigidní sortály poskytující identitu. Mají ale speciální sémantický význam z pohledu Celek-Část.
- Tudíž pro `Quantity` a `Collective` opět platí i obecná pravidla jako pro `Kind`.

# Jak vybrat relaci celek–část

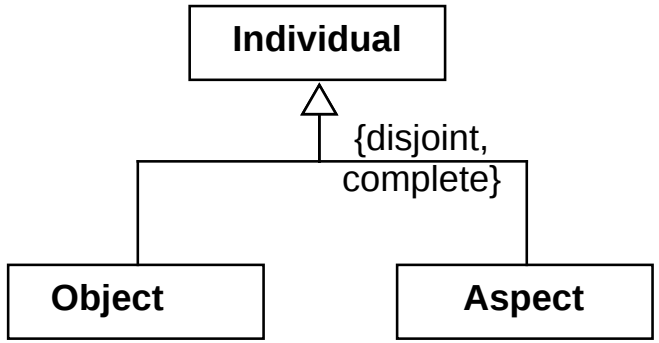
| Relace | Část / obsah → celek / kontejner | Tranzitivní? | Klíčové omezení |
|---|---|---|---|
| `componentOf` | Funkční celek → funkční celek | Obecně ne | Sdílení závisí na doméně |
| `memberOf` | Funkční celek či kolektiv → kolektiv | Ne | Celkem alespoň 2 členové |
| `subCollectionOf` | Kolektiv → kolektiv | Ano | Nejvýše 1 podkolektiv daného typu; lze sdílet |
| `containment` | Kvantita → kontejner | Ne | 1:1 v uvedeném vzoru |
| `subQuantityOf` | Kvantita → kvantita | Ano | Právě 1 na straně části; esenciální, nesdílená |

Pozn.: Na koncích mohou být podtypy příslušných poskytovatelů identity. Vzor containment 1:1 popisuje kontejner s obsahem; prázdný kontejner vyžaduje širší model.

OntoUML.org: relace celek–část a podrobná omezení

# Objekty vs. Aspekty

- Dosud jsme pracovali s typy entit, které jsou z ontologického hlediska *existenčně nezávislé*.
  - Pozn.: existenční závislost je něco jiného než relační závislost, viz dále.
- Řada entit – v reálném světě je však existenčně závislá na jiné: barva na jablku, velikost na člověku, verze na softwaru, apod.
  - takové typy entit nazýváme v UFO **Aspekty (Aspects)**.
  - Ve filosofické literatuře se objekty obvykle označují *Substances* či *Substantials* a aspekty *Accidents*, *Tropes*, *Moments*.



# Existenční závislost

- Mějme unární predikát $\epsilon$ značící existenci. Instance $x$ je *existenční závislá (existentially dependent)* na instanci $y$, značíme $ed(x, y)$ právě tehdy, když $y$ musí nutně existovat kdykoliv existuje $x$:

$$ed(x, y) =_{def} \Box(\epsilon(x) \implies \epsilon(y))$$

- Nyní můžeme definovat nezávislost dvou entit:

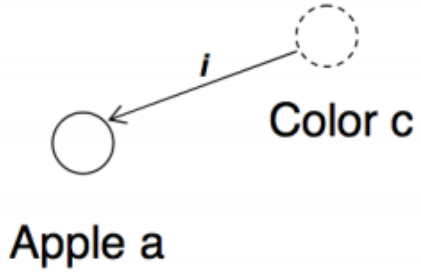
$$indep(x, y) =_{def} \neg ed(x, y) \wedge \neg ed(y, x)$$

# Objekty

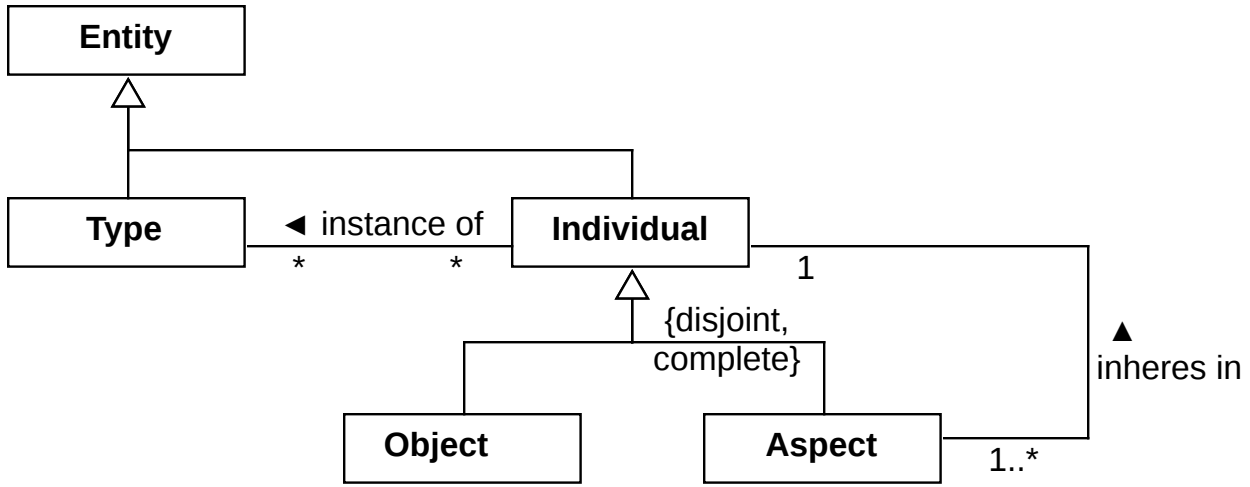
- Objekty jsou entity, které jsou nezávislé na všem s čím jsou disjunktní, tedy kromě situace, kdy Objekt je *esenciální Částí* jiného Objektu nebo je *neoddělitelná Část*.
- ⚠️ **Pozor na přetížení pojmu Objekt**:
  - Objekt jakožto pojem *objektově-orientovaného programování* = instance třídy

# Aspekty

- Aspekty jsou existenčně závislé na Objektu, kterému říkáme **nositel (bearer)**.

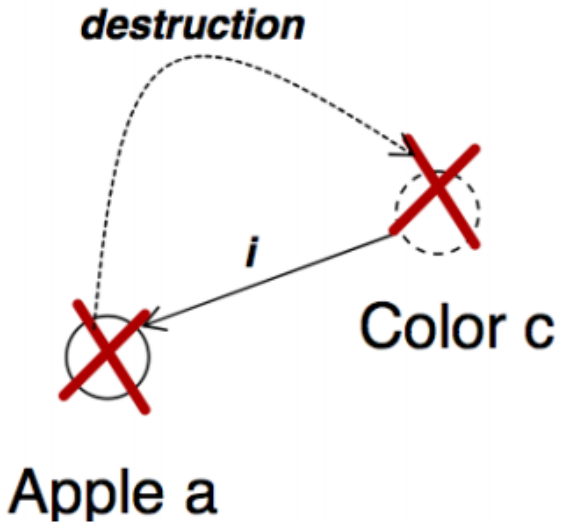



- Tato vazba existenční závislosti se nazývá **inherence**.



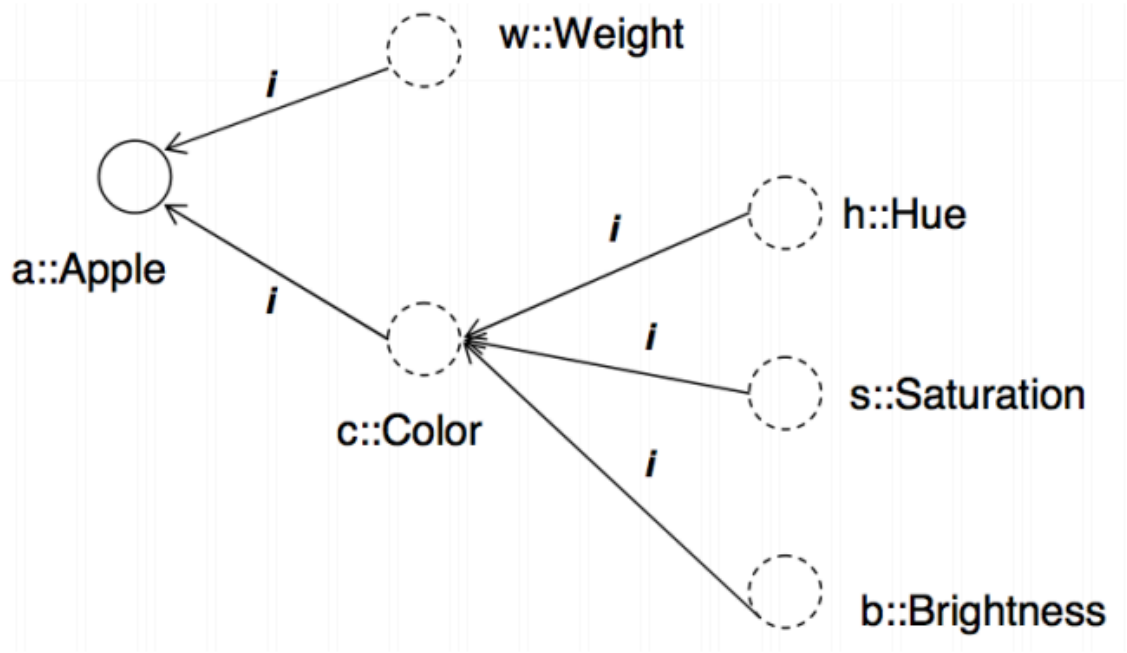
# Aspekty

- Inherence je relace **irreflexivní**, **asymetrická**, **non-tranzitivní**.
- Existenční povaha inherence navíc vyžaduje, aby při zániku Objektu zanikly i všechny jeho Aspekty:

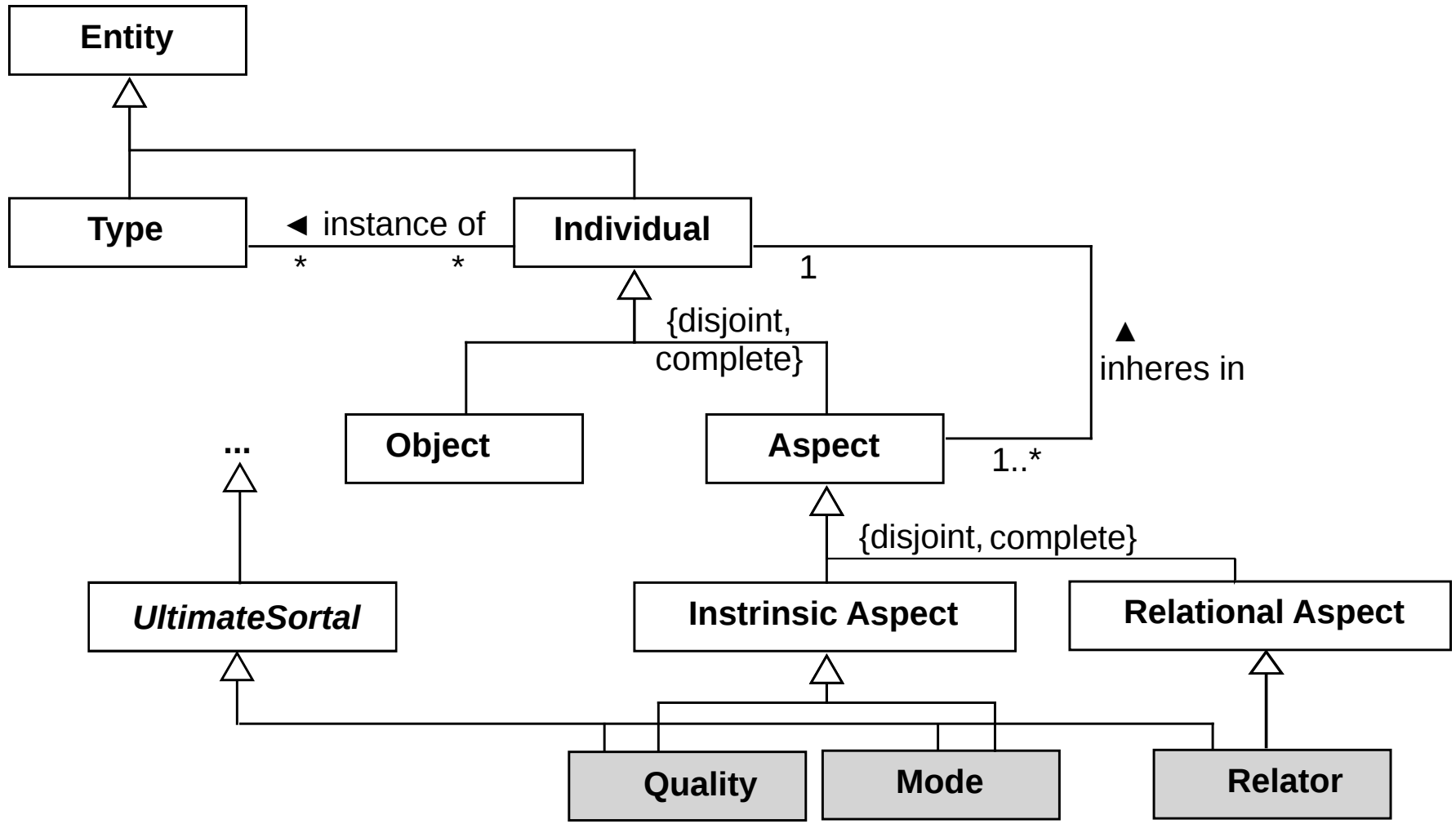


# Nepřímé Aspekty

- Nositel Aspektu může být sám Aspektem:

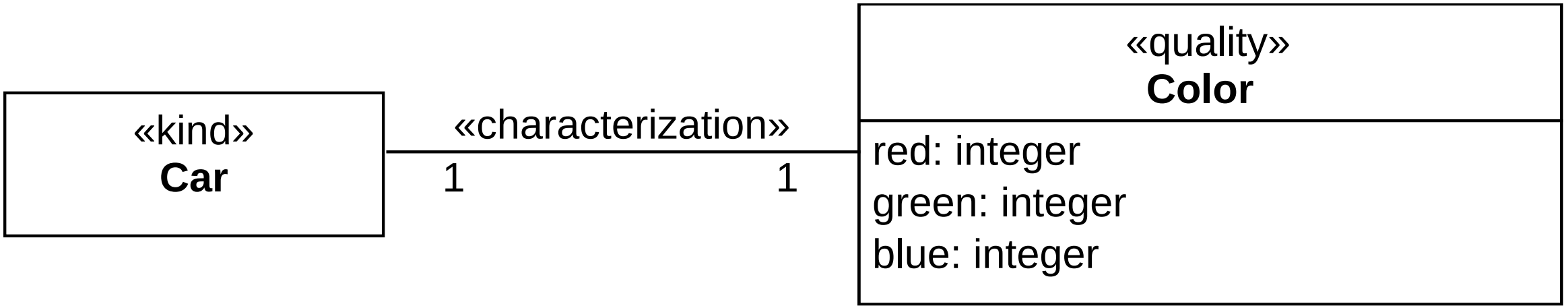


# Typy Aspektů



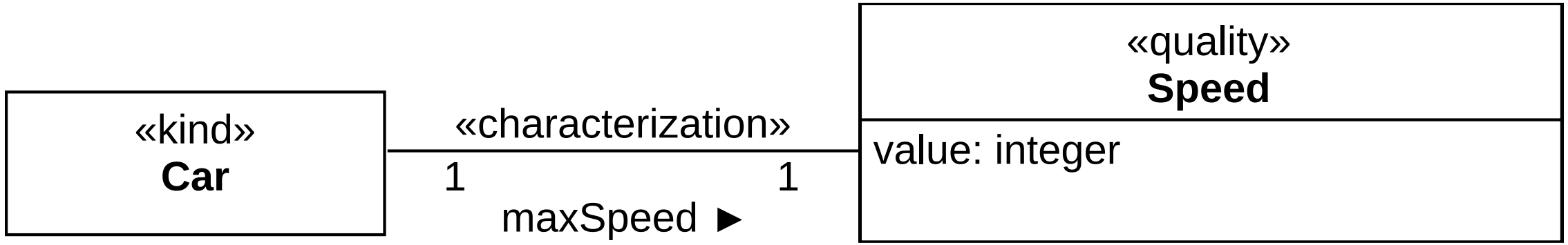
# Aspekty: Kvalita

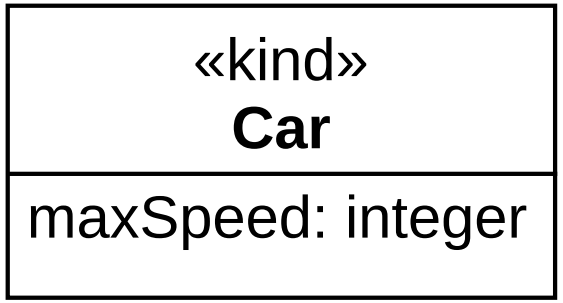
- Intrinsické aspekty se **strukturovaným prostorem hodnot**: hmotnost, barva, peněžní hodnota; také nominální kvality, např. jméno.
- `Quality` není stereotyp celek–část `Quantity`.
- `«characterization»` spojuje Kvalitu či Mód s **právě jedním nositelem** (multiplicita `1` na straně nositele).



# Aspekty: Kvalita

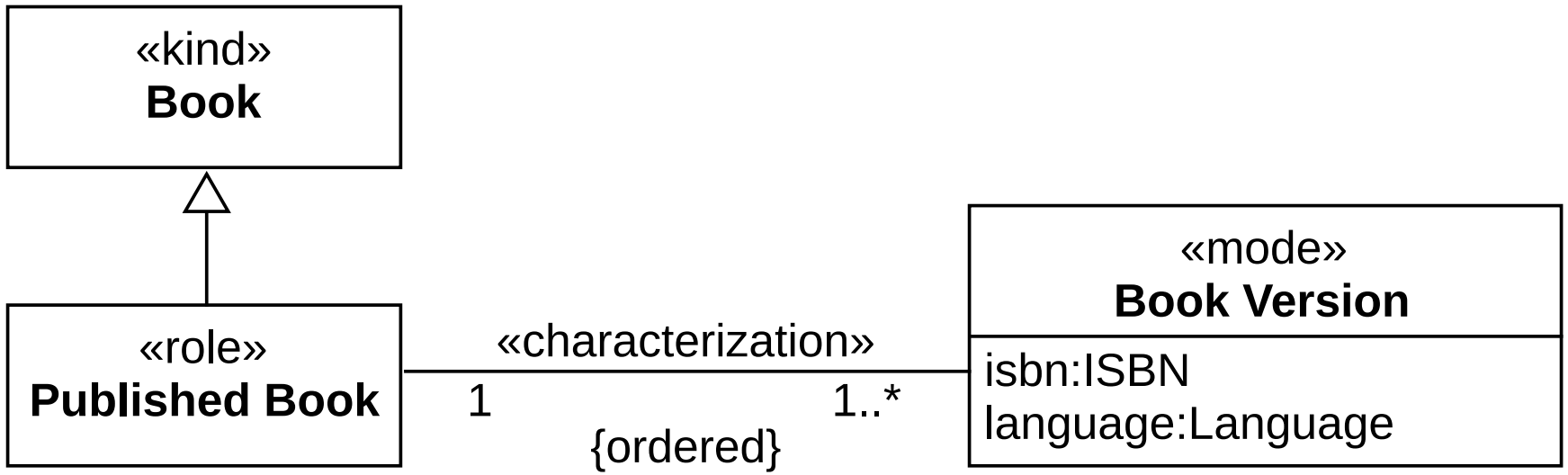
- Atributy jsou v OntoUML de-facto "syntactic sugar" pro Kvality.
- Pro úsporu místa je ale často používáme a typicky volíme samostatnou Kvalitu jen pokud potřebujeme modelovat její případné další atributy a vazby kromě toho, že reprezentuje *hodnotu*.



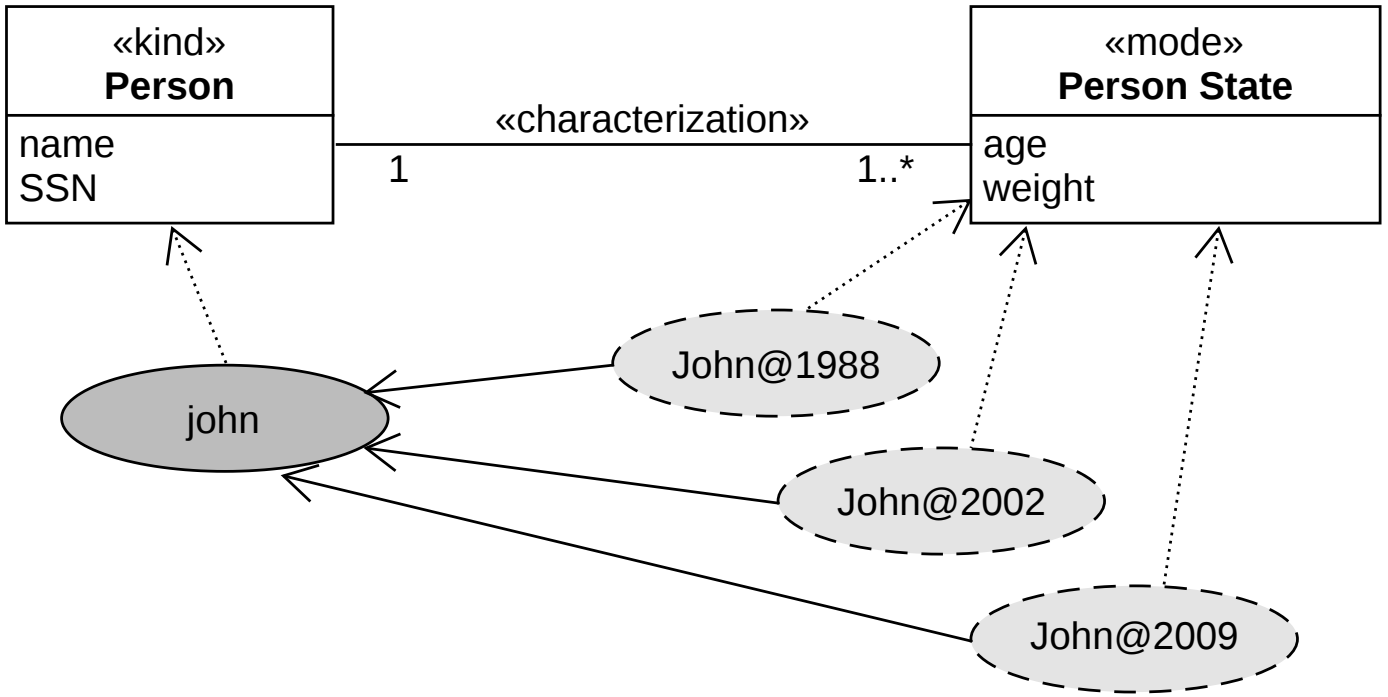
# Aspekty: Mód

- Módy jsou intrinsické aspekty **bez strukturovaného prostoru hodnot**, např. dovednost či konkrétní nemoc. Měřitelné vlastnosti módu z něj ještě nedělají Kvalitu.
- Módy jsou analogické *slabým entitám* v ER modelování.



# Aspekty: Mód

- Pomocí Módů často reprezentujeme stavy Objektů:




Pozn.: «Phase» jsme přirovnávali ke "stavům", ale zde je vidět, že pojem "stav" je opět ontologicky nepřesný a může znamenat vícero věcí.

# Shrnutí aspektů

- Existenčně závislé entity.
- Patří mezi ně «Relator», «Quality» a «Mode».
- **Kvalita:** aspekt se strukturovaným prostorem hodnot; **Mód:** aspekt bez něj. Samotná měřitelnost nerozhoduje.
- Atributy mohou zkracovat zápis kvalit. Samostatná Kvalita je užitečná pro vlastní vlastnosti, vazby či různé stupnice.

Definice a příklady: Quality · Mode — OntoUML.org

# Časté otázky

## „Když má auto povinně motor, znamená to, že ho nesmím vyměnit?“

Povinná část znamená požadavek na nějaký motor; esenciální část váže existenci celku na konkrétní motor. Navíc musíme vyjasnit, zda naše doména připouští auto dočasně bez motoru během opravy — pak by povinnost pro všechna auta byla příliš silná.

# Časté otázky

## „Musím každou vlastnost kreslit jako samostatnou třídu Quality?“

Nemusíme: přednáška používá atributy jako úsporný zápis kvalit. Samostatná kvalita se hodí, když potřebujeme zachytit další její vlastnosti nebo vztahy. Přitom rozlišujeme samotnou kvalitu, například hmotnost auta, od její číselné hodnoty a od záznamu měření.

# Konec přednášky